โซน A · 8 ตาราง | โซน B · 9 ตาราง | โซน C · 4 ตาราง

พจนานุกรมข้อมูล (Data Dictionary)

ระบบประกันรายได้ SGI / SBP Mall — ตารางที่ต้องสร้างใหม่ทั้งหมด

21 ตาราง · 270 คอลัมน์ · อธิบายครบทุกคอลัมน์

ขอบเขต	21 ตารางใหม่ (โซน A 8 · B 9 · C 4) · 270 คอลัมน์
ตอบคำถาม	ตารางนี้มาจากไหน · ใช้ทำอะไร · ทำไมต้องมี · แต่ละคอลัมน์เก็บอะไรไว้ใช้ทำอะไร
ไม่รวม	ตารางของระบบ SBP เดิมที่ SGI แค่อ่าน — fcs_qssi_score · store/mas_store/sevenshop · common_code · mas_param · business_user · email_template · sps_store.workflow_* (13 ตาราง)
อ่านคู่กับ	LLDD-Database (DDL · index · transaction) · database.md (ตารางต้นทางตอน migrate)
ที่มาของเนื้อหา	tools/lldd_db_dictionary.py — รายชื่อคอลัมน์ดึงสดจาก DDL ทุกครั้งที่ build

สารบัญ

โซน	ตาราง	ทำหน้าที่อะไร	คอลัมน์
A	sgi_fgi_impact_processes	hub ของ "หนึ่งรอบชดเชย" (ร้าน × งวดเดือน)	19
A	sgi_fgi_impact_stores	คู่ "ร้านถูกกระทบ × ร้านเปิดใหม่" ของแต่ละรอบ	21
A	sgi_fgi_impact_sales_summaries	สรุปยอดขายก่อน/หลังร้านใหม่เปิด (หัวของยอดขาย)	9
A	sgi_fgi_new_store_compensations	ค่าชดเชยฝั่งร้านเปิดใหม่ รายงวด (ลูกของค่าชดเชยฝั่งร้านถูกกระทบ)	12
A	sgi_sales_transactions	ยอดขายรายวันจาก IAS (ลูกของตารางสรุป)	10
A	sgi_fgi_impact_competitors	ร้านคู่แข่งที่กระทบ (ระดับ pipeline)	14
A	sgi_fgi_impact_compensations	ยอดชดเชยรายงวดของแต่ละรอบ	18
A	sgi_interface_transactions	สมุดกลางของการรับ–ส่งข้อมูลกับระบบภายนอก (tracking + transactional outbox)	26
B	sgi_compensation_documents	เอกสารประกันรายได้ (หัวเอกสาร) — ตารางกลางของทั้งโซน B	31
B	sgi_document_new_stores	ร้านเปิดใหม่ในเอกสาร + การปันส่วน %ชดเชย	9
B	sgi_document_competitors	ร้านคู่แข่งในเอกสาร (ระดับสาขา)	18
B	sgi_document_external_factors	ปัจจัยภายนอกที่ยกมาในเอกสาร	8
B	sgi_consideration_logs	ประวัติการพิจารณาทุกขั้น (ใครตัดสินอะไร เมื่อไร)	10
B	sgi_document_attachments	ไฟล์แนบของเอกสาร (เก็บเฉพาะ metadata ตัวไฟล์อยู่บน S3)	14
B	sgi_compensation_histories	ประวัติการชดเชยรายร้าน (ข้ามเอกสาร)	8
B	sgi_document_cost_details	ยอดชดเชยแยกรายเดือน × รายร้านเปิดใหม่	10
B	sgi_document_running_numbers	ตัวนับเลขเอกสารต่อปี	4
C	sgi_impacted_stores	ทะเบียนร้านที่ถูกกระทบ (ร้าน SP ที่อยู่ในขอบเขตประกันรายได้)	8
C	sgi_external_factors	master ปัจจัยภายนอก	5
C	sgi_section_owners	ทะเบียนผู้รับผิดชอบต่อขั้น (× ภาค)	10
C	sgi_competitors	master แบรนด์คู่แข่ง	6

โซน A FGI/FCS Impact Pipeline และ interface ภายนอก — 8 ตาราง

sgi_fgi_impact_processes

19 คอลัมน์

hub ของ "หนึ่งรอบชดเชย" (ร้าน × งวดเดือน)

มาจากไหน	FGI/FCS · migrate จาก ORA `FGI_IMPACT_STORE_ON_PROCESS` (PK `IMPACT_PROCESS_ID` · sequence `SEQ_FGI_IMPACT_PROCESS`)
ใช้ทำอะไร	เป็น แถวแม่ของทุกอย่างในรอบชดเชยหนึ่งรอบ — 1 แถว = ร้านหนึ่งร้าน ในงวดหนึ่งงวด ตารางลูกทั้งหมดในโซน A ห้อยจาก `id` ของตารางนี้
ทำไมต้องมี	ถ้าไม่มีแถวแม่ ข้อมูลรายรอบ (ร้านเปิดใหม่ · ยอดขาย · คู่แข่ง · ยอดชดเชย) จะไม่มีอะไรมัดรวมกัน และจะตอบไม่ได้ว่า "รอบนี้ประมวลถึงไหนแล้ว" · `id` ของตารางนี้คือต้นทางของ data spine impact_process_id → doc_no → transaction_id → approver_id
ใครเขียน	Job 2 (สร้างรอบ) · Job 4/5 (อัปเดตสถานะ) · Job 8b (ตั้ง `workflow_generation_status`)
ใครอ่าน	Job 6/7/8/9 · `GET /sgi/workflow/summary` · รายงานสรุปสถานะ

กติกาที่ต้องรู้ก่อนเขียนโค้ด

- UNIQUE (`impacted_store_code`, `impact_month`) — ร้านหนึ่งร้านมีได้รอบเดียวต่อหนึ่งงวด กันสร้างซ้ำตอน rerun
- `workflow_generation_status` อยู่ที่ตารางนี้ ที่เดียว — ห้ามมีธงซ้ำที่ `sgi_fgi_impact_stores`

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
id	BIGSERIAL	กุญแจแทน (surrogate key) ที่ฐานข้อมูลออกให้เอง เรียงเพิ่มทีละ 1	ใช้เป็น PK และเป็นเป้าของ FK จากตารางลูก · ห้ามเอาไปโชว์ผู้ใช้ (ผู้ใช้เห็น doc_no หรือรหัสร้านแทน) · เหตุผลที่ไม่ใช้ business key เป็น PK คือ business key ของระบบเดิมเปลี่ยนได้และบางตารางมีคีย์ผสม 3-4 คอลัมน์
impacted_store_code	VARCHAR(5)	รหัสร้านที่ถูกกระทบในรอบนี้	FK ไป `sgi_impacted_stores` · ครึ่งหนึ่งของ business key
impact_month	CHAR(7)	งวดเดือนที่กระทบ รูปแบบ `YYYY-MM` (ค.ศ.)	อีกครึ่งของ business key · ใช้เป็นตัวกรองหลักของทุกรายงาน
impact_year	INTEGER	ปี ค.ศ. ของงวด แยกออกมาเป็นตัวเลข	มีไว้เพื่อ กรอง/จัดกลุ่มรายปีโดยไม่ต้อง substring `impact_month` ซึ่งทำให้ index ใช้ไม่ได้
process_status	VARCHAR(30)	สถานะการประมวลผลของรอบ (ขั้นตอน pipeline ถึงไหนแล้ว)	ใช้ให้ job ที่รันทีหลังรู้ว่างานก่อนหน้าเสร็จหรือยัง และใช้ตอน rerun ว่าจะข้ามแถวไหน
action_status	VARCHAR(30)	สถานะการตัดสินของรอบ (`Y` = ดำเนินการต่อ · `W` = รอ · `N` = ไม่เข้าเกณฑ์)	เป็น gate ว่ารอบนี้จะถูกสร้างเป็นเอกสารหรือไม่ — Job 8 อ่านค่านี้
last_compensation_amount	NUMERIC(14,2)	ยอดชดเชยของรอบล่าสุดของร้านนี้	ใช้เทียบว่ายอดงวดนี้เป็น 0 ติดกันกี่เดือน เพื่อบังคับกฎ "เดือนที่ 4 หยุดชดเชย"
workflow_generation_status	CHAR(1)	ผลของ Gen Flow Gate: `W` = ข้อมูลยังไม่พร้อม (ลองใหม่ได้) · `Y` = เปิด workflow สำเร็จ · `N` = ไม่เข้าเกณฑ์ถาวร	คอลัมน์เดียวที่ Job 8b ใช้ตัดสินและกันเปิด workflow ซ้ำ · `W` ต่างจาก `N` ตรงที่ `W` จะถูกหยิบมาลองใหม่รอบหน้า ส่วน `N` จบถาวร
last_compensate_seq	INTEGER	ลำดับรอบชดเชยล่าสุด (นับรวมทุกครั้ง)	ใช้ต่อเลขลำดับให้แถวใหม่ใน `sgi_fgi_impact_compensations`

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
last_compensate_seq_no	INTEGER	เลขลำดับย่อยของรอบล่าสุด	ใช้คู่กับ `last_compensate_seq` ตอนรอบเดียวมีหลายงวดย่อย
start_compensate_month	CHAR(7)	งวดเดือนแรกที่เริ่มชดเชยร้านนี้ (`YYYY-MM` ค.ศ.)	ใช้นับว่าตอนนี้เป็น "เดือนที่เท่าไรของการชดเชย" ซึ่งเป็นหัวใจของกฎเดือน 1–3 / เดือน 4
start_compensate_year	INTEGER	ปี ค.ศ. ของงวดแรก	แยกไว้เพื่อ query รายปีโดยไม่ substring
end_compensate_month	CHAR(7)	งวดเดือนสุดท้ายที่ชดเชย	ตั้งเมื่อรอบจบ (หยุดชดเชย) · ใช้ปิดรอบไม่ให้ job หยิบมาทำอีก
end_compensate_year	INTEGER	ปี ค.ศ. ของงวดสุดท้าย	คู่กับ `end_compensate_month`
flag_action	CHAR(1)	ธงบอกว่ารอบนี้ถูกดำเนินการแล้วหรือยัง (`Y`/`N`)	ตัวกัน job หยิบแถวเดิมซ้ำภายในรอบการรันเดียวกัน
datasource	VARCHAR(5)	ระบบต้นทางของแถว เช่น `ALM` (ALLMAP) · `FGI`	ใช้แยกว่าข้อมูลมาจาก pipeline อัตโนมัติหรือคีย์มือ ตอนสืบหาสาเหตุข้อมูลผิด
created_at	TIMESTAMP	เวลาที่แถวนี้ถูกสร้าง (ตั้งอัตโนมัติด้วย `DEFAULT CURRENT_TIMESTAMP`)	ใช้เรียงลำดับเหตุการณ์ · เป็นฐานคำนวณอายุงานของ watchdog · ใช้สืบย้อนตอน incident ว่าแถวเกิดตอนไหน — ห้ามให้แอปเขียนทับ
updated_by	VARCHAR(30)	ผู้แก้ไขล่าสุด (รูปแบบเดียวกับ `created_by`)	ใช้ตอบคำถาม "ใครแก้ล่าสุด" ตอนข้อมูลเพี้ยน
updated_at	TIMESTAMP	เวลาที่แถวนี้ถูกแก้ล่าสุด	ใช้ตรวจว่าข้อมูล master ถูกแก้เมื่อไร และใช้เป็นตัวช่วยตอน sync/รีเฟรช cache ฝั่ง FE

sgi_fgi_impact_stores

21 คอลัมน์

คู่ "ร้านถูกกระทบ × ร้านเปิดใหม่" ของแต่ละรอบ

มาจากไหน	FGI/FCS · migrate จาก ORA `FGI_IMPACT_STORE` (PK `IMPACT_STORE_ID` · business key `STORECODE_I + MONTH + YEAR`)
ใช้ทำอะไร	เก็บว่ารอบนี้ร้าน SP ถูกกระทบจาก ร้านเปิดใหม่ร้านไหนบ้าง พร้อมระยะทางและ %ชดเชยที่ปันให้แต่ละร้าน
ทำไมต้องมี	ร้านหนึ่งร้านอาจถูกกระทบจากร้านเปิดใหม่หลายร้านพร้อมกัน · ยอดชดเชยต้อง ปันส่วนตาม % ให้ครบ 100% พอดี ตารางนี้คือที่เก็บการปันส่วนนั้น
ใครเขียน	Job 2 (สร้างคู่จาก ALLMAP) · Job 4/5 (เติม %/ยอด) · Job 9 (sync ไปเอกสาร)
ใครอ่าน	Job 6/8/9 · `sgi_document_new_stores` (ปลายทางที่ sync ไป)

กติกาที่ต้องรู้ก่อนเขียนโค้ด

- ผลรวม `forecast_compensate_percent` ของทุกแถวใน `impact_process_id` เดียวกัน ต้องเท่ากับ 100 พอดี — ตรวจก่อน commit เสมอ
- ▲ ตารางนี้ ไม่มีคอลัมน์ `impact_year` (มีแต่ `impact_month`) — เคยมีคนเดาว่ามีแล้วเขียน SQL พัง

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
id	BIGSERIAL	กุญแจแทน (surrogate key) ที่ฐานข้อมูลออกให้เอง เรียงเพิ่มทีละ 1	ใช้เป็น PK และเป็นเป้าของ FK จากตารางลูก · ห้ามเอาไปโชว์ผู้ใช้ (ผู้ใช้เห็น doc_no หรือรหัสร้านแทน) · เหตุผลที่ไม่ใช้ business key เป็น PK คือ business key ของระบบเดิมเปลี่ยนได้และบางตารางมีคีย์ผสม 3-4 คอลัมน์

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
impact_process_id	BIGINT	FK ไปแถวแม่ในรอบเดียวกัน	มัดทุกคู่เข้ากับรอบชดเชยรอบเดียว
impacted_store_code	VARCHAR(5)	รหัสร้าน SP ที่ถูกกระทบ	เก็บซ้ำจากแถวแม่เพื่อให้ query ระดับคู่ไม่ต้อง join ขึ้นไป
new_store_code	VARCHAR(5)	รหัสร้าน 7-Eleven ที่เปิดใหม่และเป็นต้นเหตุ	อีกครึ่งของคู่ · ใช้ join ไป master ร้านของระบบเดิมเพื่อเอาชื่อ/พิกัด
impact_month	CHAR(7)	งวดเดือนของคู่นี้ (YYYY-MM ค.ศ.)	ใช้ยืนยันว่าแถวลูกอยู่งวดเดียวกับแถวแม่
distance_km	NUMERIC(8,3)	ระยะทางจากร้านเปิดใหม่ถึงร้าน SP (กม. ทศนิยม 3 ตำแหน่ง)	ค่าที่ ALLMAP คัดมาแล้ว — SGI ไม่กรองรัศมี 1/2 กม. ซ้ำ (มติ 2026-10-05 · เชื่อผลของ ALLMAP) · เก็บไว้แสดงและตรวจย้อนหลัง
allmap_url	VARCHAR(500)	ลิงก์แผนที่ ALLMAP ของคู่ร้านนี้ (URL_EMBEDDED ที่ ALLMAP ส่งมาในไฟล์)	คนขนค่าจาก Job 2 ไปถึง Job 8 (ปิดข้อค้าง 2.14 เมื่อ 2026-09-29) · Job 8 ต่อ พารามิเตอร์ base64 (storeCodeI/storeCodeN/period/uname · รูปเดียวกับลิงก์ของเอกสารเดิมที่ ALLMAP อ่านได้) แล้วเขียนลง sgi_compensation_documents.allmap_url ให้ปุ่ม 'แผนที่ AllMap' ใช้ · ไม่มีคอลัมน์นี้ = ค่าที่ ALLMAP ส่งมาถูกทิ้งทุกงวดและปุ่มว่างตลอดไป
branchtype_i	VARCHAR(10)	ประเภทร้านของร้านที่ถูกกระทบ ตามที่ ALLMAP ส่งมา ณ ตอนนำเข้า	Job 2 ใช้ตัดสิน P/N แบบเดียวกับระบบเดิม (มติ 2026-10-05) · เก็บเป็น snapshot เพื่อให้ตรวจย้อนหลังได้ว่าตัดสินด้วยค่าอะไร · แถวจาก STA ใช้ master แทน
branchtype_n	VARCHAR(10)	ประเภทร้านของร้านเปิดใหม่ ตามที่ ALLMAP ส่งมา ณ ตอนนำเข้า	ใช้กับกฎ 'ร้านเปิดใหม่เป็นประเภท F = ตัดทิ้ง' · snapshot เหตุผลเดียวกับ branchtype_i
opendate_i	DATE	วันเปิดร้านที่ถูกกระทบ ตามที่ ALLMAP ส่งมา ณ ตอนนำเข้า	Job 4 ใช้คัดร้านที่ถึงเวลาขอยอดขาย (มติ 2026-10-05) · แถว STA ใช้ master แทนแบบระบบเดิม
opendate_n	DATE	วันเปิดร้านเปิดใหม่ ตามที่ ALLMAP ส่งมา ณ ตอนนำเข้า	Job 4 ใช้คัดร้าน + เขียนลงไฟล์ AMS06001O ฟิลด์ 2 · เหตุผลเดียวกับ opendate_i
verify_status	CHAR(1)	ผลการตรวจสอบคู่นี้ (Y/N/ว่าง)	ธงว่าคู่นี้ผ่านการตรวจแล้วหรือยัง ก่อนเอาไปคำนวณต่อ
created_by	VARCHAR(10)	ผู้สร้างแถว — x-user-id ของผู้ใช้จริง หรือชื่อ job (เช่น JOB-8) เมื่อ batch เป็นคนสร้าง	ใช้ตอบคำถาม "ใครทำให้เกิดแถวนี้" · แทนระบบ audit ที่ตัดทิ้งไปเมื่อ 2026-08-07 จึงเป็นร่องรอยชั้นเดียวที่เหลือ
updated_by	VARCHAR(10)	ผู้แก้ไขล่าสุด (รูปแบบเดียวกับ created_by)	ใช้ตอบคำถาม "ใครแก้ล่าสุด" ตอนข้อมูลเพี้ยน
created_at	TIMESTAMP	เวลาที่แถวนี้ถูกสร้าง (ตั้งอัตโนมัติด้วย DEFAULT CURRENT_TIMESTAMP)	ใช้เรียงลำดับเหตุการณ์ · เป็นฐานคำนวณอายุงานของ watchdog · ใช้สืบย้อนตอน incident ว่าแถวเกิดตอนไหน — ห้ามให้แอปเขียนทับ
sales_request_status	CHAR(1)	สถานะการขอข้อมูลยอดขายจาก IAS: W รอส่ง · P ส่งแล้วรอผล · Y ได้ผลแล้ว · E ผิดพลาด	เป็นตัวคุมลำดับ Job 4 (ส่งคำขอ) → Job 5 (รับผล) · เพิ่มค่า E เมื่อ 2026-09-02 ปิดช่องว่าง G1 เพราะเดิมไม่มีที่เก็บผล DENY
forecast_compensate_percent	NUMERIC(7,4)	%ชดเชยที่ปันให้ร้านเปิดใหม่ร้านนี้ (ค่าคาดการณ์)	ตัวคูณหลักของการคำนวณยอด · ผลรวมทุกแถวต้อง = 100
adjust_compensate_percent	NUMERIC(7,4)	%ชดเชยหลังปรับแก้โดยผู้ใช้	ใช้แทนค่า forecast เมื่อมีการปรับ · nullable = ยังไม่ปรับ
forecast_compensation_amount	NUMERIC(14,2)	ยอดเงินชดเชยที่คาดการณ์สำหรับคู่นี้	= ยอดรวมของรอบ × forecast_compensate_percent
adjust_compensation_amount	NUMERIC(14,2)	ยอดเงินชดเชยหลังปรับ	ค่าที่ใช้จริงเมื่อมีการปรับแก้
updated_at	TIMESTAMP	เวลาที่แถวนี้ถูกแก้ล่าสุด	ใช้ตรวจว่าข้อมูล master ถูกแก้เมื่อไร และใช้เป็นตัวช่วยตอน sync/รีเฟรช cache ฝั่ง FE

sgi_fgi_impact_sales_summaries

9 คอลัมน์

สรุปยอดขายก่อน/หลังร้านใหม่เปิด (หัวของยอดขาย)

มาจากไหน	FGI/FCS · migrate จาก ORA `FGI_IMPACT_STORE_SALES` (key `STORECODE_I + MONTH + YEAR`)
ใช้ทำอะไร	เก็บ ผลสรุปการเปรียบเทียบยอดขายก่อนและหลังร้านใหม่เปิด — อัตราเติบโตก่อน/หลังและผลต่าง ซึ่งเป็นตัวตัดสินว่าร้านนี้ "ยอดตกจริง" หรือไม่
ทำไมต้องมี	เกณฑ์ธุรกิจข้อหลักคือ growth ต่างกัน ≤ −10% ถึงจะเข้าข่ายชดเชย · ต้องมีที่เก็บผลคำนวณนี้แยกจากยอดขายรายวัน เพราะ API และรายงานอ่านค่าสรุปบ่อยกว่ารายวันมาก
ใครเขียน	Job 5 (ImportImpactSaleFromIAS) — คำนวณจากยอดรายวันที่รับมา
ใครอ่าน	Job 8 (สร้างเอกสาร) · Job 8b (Gen Flow Gate) · หน้าเอกสาร (กราฟ/ตารางยอดขาย)

กติกาที่ต้องรู้ก่อนเขียนโค้ด

- 1 แถวต่อ 1 `impact_process_id` — เป็นหัวของ `sgi_sales_transactions` แบบ 1:N

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
id	BIGSERIAL	กุญแจแทน (surrogate key) ที่ฐานข้อมูลออกให้เอง เรียงเพิ่มทีละ 1	ใช้เป็น PK และเป็นเป้าของ FK จากตารางลูก · ห้ามเอาไปโชว์ผู้ใช้ (ผู้ใช้เห็น doc_no หรือรหัสร้านแทน) · เหตุผลที่ไม่ใช้ business key เป็น PK คือ business key ของระบบเดิมเปลี่ยนได้และบางตารางมีคีย์ผสม 3-4 คอลัมน์
impact_process_id	BIGINT	FK ไปรอบชดเชย	มัดยอดขายเข้ากับรอบ
total_working_days	INTEGER	จำนวนวันทำการที่มีข้อมูลยอดขายจริง	เกณฑ์คือ ต้องครบ 60 วัน · ไม่ครบ = ร้านขึ้นธงแดง "ยอดขายไม่ครบ 60 วัน" และเข้าเงื่อนไข pre-accept
growth_rate_before	NUMERIC(9,4)	อัตราเติบโตยอดขาย ก่อน ร้านใหม่เปิด (%)	ฐานเปรียบเทียบ
growth_rate_after	NUMERIC(9,4)	อัตราเติบโตยอดขาย หลัง ร้านใหม่เปิด (%)	ค่าที่เอามาเทียบกับฐาน
growth_rate_diff	NUMERIC(9,4)	ผลต่าง = after − before (%)	ตัวตัดสินเกณฑ์หลัก: ต้อง ≤ −10 ถึงเข้าข่ายชดเชย · ถ้าเป็น NULL แปลว่าข้อมูลยังไม่พร้อม Gate จะคง `W` ไว้
sales_status	CHAR(1)	ธงว่าข้อมูลยอดขายพร้อมใช้หรือยัง (`Y`/`N`)	เป็นเงื่อนไขหนึ่งของ Gen Flow Gate — ค่าอื่นที่ไม่ใช่ `Y`/`N` ให้คง `W` และคืน 422
updated_by	VARCHAR(30)	ผู้แก้ไขล่าสุด (รูปแบบเดียวกับ `created_by`)	ใช้ตอบคำถาม "ใครแก้ล่าสุด" ตอนข้อมูลเพี้ยน
updated_at	TIMESTAMP	เวลาที่แถวนี้ถูกแก้ล่าสุด	ใช้ตรวจว่าข้อมูล master ถูกแก้เมื่อไร และใช้เป็นตัวช่วยตอน sync/รีเฟรช cache ฝั่ง FE

sgi_fgi_new_store_compensations

12 คอลัมน์

ค่าชดเชยฝั่งร้านเปิดใหม่ รายงวด (ลูกของค่าชดเชยฝั่งร้านถูกกระทบ)

มาจากไหน	FGI/FCS · migrate จาก ORA `FGI_NEW_STORE_COMPENSATE` (23,628 แถวจริง)
ใช้ทำอะไร	แตกยอดชดเชยของงวดหนึ่ง ออกเป็น รายร้านเปิดใหม่ พร้อมสัดส่วน %ที่แต่ละร้านรับผิดชอบ
ทำไมต้องมี	ร้านถูกกระทบหนึ่งร้านอาจโดนร้านใหม่หลายสาขาพร้อมกัน · เงินชดเชยต้องกระจายตามสัดส่วน และ ผลรวม %ของทุกร้านใหม่ในงวดเดียวต้องได้ 100% — ถ้าไม่แตกเป็นแถว ตรวจจกติกานี้ไม่ได้
ใครเขียน	Job 6 — insert ตอนเปิดรอบชดเชยใหม่ (mutation ที่ 6)
ใครอ่าน	Job 6 — ประกอบ payload ส่ง STA · หน้าจอเอกสาร (ตารางแบ่ง %ชดเชย)

กติกาที่ต้องรู้ก่อนเขียนโค้ด

- 1 แถว = 1 (งวด × ร้านเปิดใหม่) — ไม่ใช่ 1 แถวต่อคู่ร้าน · คู่ร้านเดิมได้แถวใหม่ทุกงวดที่ยังชดเชยต่อเนื่อง
- • ห้ามย้ายข้อมูลนี้ไปไว้บน `sgi_fgi_impact_stores` — ตารางนั้นไม่มีมิติงวด งวดใหม่จะทับงวดเก่าแล้วตรวจย้อนหลังไม่ได้ว่างวดไหนจ่ายเท่าไร
- ผลรวม `forecast_percent` ของทุกแถวที่มี `impact_compensation_id` เดียวกัน ต้องเท่ากับ 100

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
id	BIGSERIAL	กุญแจแทน (surrogate key) ที่ฐานข้อมูลออกให้เอง เรียงเพิ่มทีละ 1	ใช้เป็น PK และเป็นเป้าของ FK จากตารางลูก · ห้ามเอาไปโชว์ผู้ใช้ (ผู้ใช้เห็น doc_no หรือรหัสร้านแทน) · เหตุผลที่ไม่ใช้ business key เป็น PK คือ business key ของระบบเดิมเปลี่ยนได้และบางตารางมีคีย์ผสม 3-4 คอลัมน์
impact_compensation_id	BIGINT	FK ไปค่าชดเชยฝั่ง I ของงวดนั้น	มัดยอดรายร้านใหม่เข้ากับงวด · `ON DELETE CASCADE` เพราะลูกไม่มีความหมายถ้าไม่มีงวด
impact_store_id	BIGINT	FK ไปแถวคู่ร้าน (ร้านถูกกระทบ ↔ ร้านเปิดใหม่)	ORA `IMPACT_STORE_ID` · ตามรอยกลับไปหาต้นเรื่องว่าคู่นี้มาจาก import รอบไหน
new_store_code	VARCHAR(5)	รหัสร้านเปิดใหม่ 5 หลัก	ฟิลด์ที่ 2 ของ payload ที่ส่ง STA
forecast_amount	NUMERIC(14,2)	ยอดชดเชยที่ประมาณการไว้ (บาท)	ค่าตั้งต้นก่อนผู้พิจารณาปรับ
forecast_percent	NUMERIC(7,4)	สัดส่วน %ที่ร้านใหม่นี้รับผิดชอบ (ประมาณการ)	รวมทุกแถวในงวดเดียวต้องได้ 100
adjust_amount	NUMERIC(14,2)	ยอดชดเชยหลังปรับ (บาท)	ค่าที่ผู้พิจารณาแก้ — ว่างแปลว่ายังไม่ถูกปรับ
adjust_percent	NUMERIC(7,4)	สัดส่วน %หลังปรับ	รวมทุกแถวในงวดเดียวต้องได้ 100 เช่นกัน
created_by	VARCHAR(100)	ผู้สร้างแถว — `x-user-id` ของผู้ใช้จริง หรือชื่อ job (เช่น `JOB-8`) เมื่อ batch เป็นคนสร้าง	ใช้ตอบคำถาม "ใครทำให้เกิดแถวนี้" · แทนระบบ audit ที่ตัดทิ้งไปเมื่อ 2026-08-07 จึงเป็นร่องรอยชั้นเดียวที่เหลือ
created_at	TIMESTAMP	เวลาที่แถวนี้ถูกสร้าง (ตั้งอัตโนมัติด้วย `DEFAULT CURRENT_TIMESTAMP`)	ใช้เรียงลำดับเหตุการณ์ · เป็นฐานคำนวณอายุงานของ watchdog · ใช้สืบย้อนตอน incident ว่าแถวเกิดตอนไหน — ห้ามให้แอปเขียนทับ
updated_by	VARCHAR(100)	ผู้แก้ไขล่าสุด (รูปแบบเดียวกับ `created_by`)	ใช้ตอบคำถาม "ใครแก้ล่าสุด" ตอนข้อมูลเพี้ยน

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
updated_at	TIMESTAMP	เวลาที่แถวนี้ถูกแก้ล่าสุด	ใช้ตรวจว่าข้อมูล master ถูกแก้เมื่อไร และใช้เป็นตัวช่วยตอน sync/รีเฟรช cache ฝั่ง FE

sgi_sales_transactions

10 คอลัมน์

ยอดขายรายวันจาก IAS (ลูกของตารางสรุป)

มาจากไหน	FGI/FCS · migrate จาก ORA `FGI_IMPACT_STORE_SALES_TRN` (key เดียวกับหัว + SEQ) · ข้อมูลใหม่มาจากไฟล์ `AMS06001I` ที่ IAS/MIS วางบน EAI S3
ใช้ทำอะไร	เก็บ ยอดขายรายวัน ของร้านในช่วงเปรียบเทียบ แบ่งเป็น 4 หน้าต่าง × 15 วัน รอบวันที่ร้านใหม่เปิด
ทำไมต้องมี	ตัวเลขสรุป (growth) ต้องพิสูจน์ย้อนได้ว่ามาจากยอดวันไหนบ้าง · และหน้าจอเอกสารต้องโชว์ไทม์ไลน์ยอดขายให้ผู้พิจารณาดูก่อนตัดสิน
ใครเขียน	Job 5 — insert ทั้งชุดหลังอ่านไฟล์จาก S3
ใครอ่าน	`GET /sgi/document/{docNo}/sales` (ไทม์ไลน์ยอดขาย) · การคำนวณ growth ของ Job 5

กติกาที่ต้องรู้ก่อนเขียนโค้ด

- ไม่ผูกกับ `impact_process_id` โดยตรง — ผูกกับ `sales_summary_id` เท่านั้น เพื่อไม่ให้มี FK ซ้ำสองทาง
- ไฟล์ต้นทางเป็น windows-874 และปี พ.ศ. — ต้องแปลงเป็น UTF-8 และ ค.ศ. ตอนอ่านไฟล์ ห้ามเก็บ พ.ศ. ลงตารางนี้

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
id	BIGSERIAL	กุญแจแทน (surrogate key) ที่ฐานข้อมูลออกให้เอง เรียงเพิ่มทีละ 1	ใช้เป็น PK และเป็นเป้าของ FK จากตารางลูก · ห้ามเอาไปโชว์ผู้ใช้ (ผู้ใช้เห็น doc_no หรือรหัสร้านแทน) · เหตุผลที่ไม่ใช้ business key เป็น PK คือ business key ของระบบเดิมเปลี่ยนได้และบางตารางมีคีย์ผสม 3-4 คอลัมน์
sales_summary_id	BIGINT	FK ไปหัวยอดขาย	มัดยอดรายวันเข้ากับรอบผ่านหัว
txn_date	DATE	วันที่ของยอดขายแถวนี้ (ค.ศ.)	แกน X ของกราฟยอดขาย · ใช้จัดเรียงและแบ่งหน้าต่าง
window_no	SMALLINT	หมายเลขหน้าต่างเปรียบเทียบ 1–4 (แต่ละหน้าต่าง 15 วัน)	หน้าต่างก่อน/หลังวันร้านใหม่เปิด · ใช้จัดกลุ่มตอนคำนวณ growth ก่อน–หลัง
seq	SMALLINT	ตำแหน่งวันภายในหน้าต่าง 1–15 (คอลัมน์ SEQ ของตารางเดิม)	ใช้จับคู่วันเดียวกันระหว่างปีก่อนกับปีนี้ ตอนคำนวณ growth — ตารางเดิมเป็นตารางกว้างจึงจับคู่ได้ในแถวเดียว ตารางใหม่แคบจึงต้องมีคอลัมน์นี้ join
sales_amount	NUMERIC(14,2)	ยอดขายของวันนั้น (บาท)	ตัวตั้งของการคำนวณทั้งหมด
sales_diff	NUMERIC(14,2)	เปอร์เซ็นต์ที่ยอดขายวันนั้นเบี่ยงจากค่าเฉลี่ยของหน้าต่างตัวเอง = ROUND((ยอดวันนั้น − เฉลี่ยหน้าต่าง) / ยอดวันนั้น × 100)	ใช้หา outlier — หน่วยเป็นเปอร์เซ็นต์ ไม่ใช่บาท และหารด้วยยอดของวันนั้นเอง ไม่ใช่หารด้วยค่าเฉลี่ย (สูตรของระบบเดิม `ImportJdbc.java:544`)
is_outlier	BOOLEAN	ธงว่าวันนี้เป็นค่าผิดปกติหรือไม่ — เกณฑ์คือ `|sales_diff| >= 50` เปอร์เซ็นต์	เกณฑ์คือ |sales_diff| ≥ 50 (ค่าคงที่ในโค้ด · เปลี่ยนต้องขออนุมัติธุรกิจ) · วัน outlier ถูกตัดออกจากการคำนวณ growth
source_checksum	VARCHAR(64)	checksum ของไฟล์ต้นทางที่แถวนี้มาจาก	ใช้กันนำเข้าไฟล์เดิมซ้ำ และพิสูจน์ว่าข้อมูลในตารางตรงกับไฟล์ที่ IAS ส่งมาจริง

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
created_at	TIMESTAMP	เวลาที่แถวนี้ถูกสร้าง (ตั้งอัตโนมัติด้วย DEFAULT CURRENT_TIMESTAMP)	ใช้เรียงลำดับเหตุการณ์ · เป็นฐานคำนวณอายุงานของ watchdog · ใช้สืบย้อนตอน incident ว่าแถวเกิดตอนไหน — ห้ามให้แอปเขียนทับ

sgi_fgi_impact_competitors

14 คอลัมน์

ร้านคู่แข่งที่กระทบ (ระดับ pipeline)

มาจากไหน	FGI/FCS · migrate จาก ORA FGI_IMPACT_COMPETITOR (PK IMPACT_COMPETITOR_ID) · ข้อมูลใหม่ดึงจาก ALLMAP โดย Job 3
ใช้ทำอะไร	เก็บ สาขาของร้านคู่แข่ง ที่อยู่ในรัศมีกระทบของรอบนี้ ตามที่ ALLMAP ให้มา
ทำไมต้องมี	คู่แข่งเป็นข้อมูลประกอบการพิจารณา — ผู้อนุมัติต้องเห็นว่ามีคู่แข่งเปิดใหม่ในพื้นที่ด้วยหรือไม่ ก่อนตัดสินว่ายอดตกเพราะร้าน 7-Eleven ด้วยกันจริง
ใครเขียน	Job 3 (ImportImpactCompetitor) — ดึงจาก ALLMAP
ใครอ่าน	Job 7 (sync เข้าเอกสารเป็น sgi_document_competitors)

กติกาที่ต้องรู้ก่อนเขียนโค้ด

- แถวนี้เป็น ข้อมูลดิบฝั่ง pipeline · เมื่อ Job 7 sync เข้าเอกสารจะได้แถวใหม่ใน sgi_document_competitors ที่ผู้ใช้แก้ไขได้ — แก้ที่เอกสารไม่ย้อนกลับมาแก้ที่นี่

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
id	BIGSERIAL	กุญแจแทน (surrogate key) ที่ฐานข้อมูลออกให้เอง เรียงเพิ่มทีละ 1	ใช้เป็น PK และเป็นเป้าของ FK จากตารางลูก · ห้ามเอาไปโชว์ผู้ใช้ (ผู้ใช้เห็น doc_no หรือรหัสร้านแทน) · เหตุผลที่ไม่ใช้ business key เป็น PK คือ business key ของระบบเดิมเปลี่ยนได้และบางตารางมีคีย์ผสม 3-4 คอลัมน์
impact_process_id	BIGINT	FK ไปรอบชดเชย	มัดคู่แข่งเข้ากับรอบ
competitor_store_code	VARCHAR(50)	รหัสสาขาของคู่แข่งตามที่ ALLMAP ส่งมา เช่น 4832 · 789 · TD58_08	ไม่มี FK เพราะเป็นรหัสของระบบภายนอก · nullable เพราะ ALLMAP ส่งค่าว่างมาได้จริง (56 จาก 1,312 แถวในไฟล์ตัวอย่าง) · เป็นคีย์กันซ้ำระดับสาขา
brand_code	VARCHAR(30)	รหัสแบรนด์ 01–11	FK ไป sgi_competitors · nullable เพราะ map แบรนด์ไม่ได้ก็ยังต้องเก็บแถวไว้แล้วรายงานเป็น unmapped ไม่ใช่ทิ้งข้อมูล
name_th	VARCHAR(200)	ชื่อร้านคู่แข่งภาษาไทยตามที่ ALLMAP ให้มา	เก็บค่าดิบไว้เทียบ เผื่อ match แบรนด์ไม่ได้
name_en	VARCHAR(200)	ชื่อแบรนด์ภาษาอังกฤษที่ ALLMAP ส่งมา	ฟิลด์ 4 ของไฟล์ BPM06003O — ต้องส่งต่อ K2 จึงเก็บเป็น snapshot
branch_th	VARCHAR(200)	ชื่อสาขาของคู่แข่ง	ระบุว่าเป็นสาขาไหน — แบรนด์เดียวกันมีได้หลายสาขาในรัศมี
zone_code	VARCHAR(10)	โซนของสาขาคู่แข่ง	ฟิลด์ 6 ของ BPM06003O — ส่งต่อ K2
subzone_code	VARCHAR(10)	โซนย่อยของสาขาคู่แข่ง	ฟิลด์ 7 ของ BPM06003O — ส่งต่อ K2
opened_date	DATE	วันที่สาขาคู่แข่งเปิด	ใช้ดูว่าเปิดก่อนหรือหลังร้าน 7-Eleven ใหม่ — มีผลต่อการตีความสาเหตุยอดตก

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
closed_date	DATE	วันที่สาขาคู่แข่งปิด (ถ้าปิดแล้ว)	สาขาที่ปิดไปแล้วไม่ควรนับเป็นปัจจัยกระทบของงวดหลังจากนั้น
period_key	CHAR(7)	งวด YYYY-MM ที่ข้อมูลชุดนี้เป็นของ	ใช้แยกข้อมูลคู่แข่งคนละงวดออกจากกัน
updated_at	TIMESTAMP	เวลาที่แถวนี้ถูกแก้ล่าสุด	ใช้ตรวจว่าข้อมูล master ถูกแก้เมื่อไร และใช้เป็นตัวช่วยตอน sync/รีเฟรช cache ฝั่ง FE
competitor_key	VARCHAR(300)	คีย์ธุรกิจของสาขาคู่แข่ง — generated: ใช้ competitor_store_code ถ้ามี ไม่งั้น fallback เป็น name:<ชื่อสาขา>	ALLMAP ส่งสาขาที่ไม่มีรหัสมาได้ (ORA COMPET_ID ไม่มี NOT NULL) · ใช้รหัสอย่างเดียวเป็นคีย์ไม่ได้: NULLS DISTINCT ทำให้แถวพอกทุกรอบ · NULLS NOT DISTINCT ยุบหลายสาขาเหลือแถวเดียว = ข้อมูลหาย (ร้าน 07109 มีแฟมิลี่มาร์ท 80 สาขา) · คอลัมน์นี้จึงแยกสาขาด้วยชื่อเมื่อไม่มีรหัส

sgi_fgi_impact_compensations

18 คอลัมน์

ยอดชดเชยรายงวดของแต่ละรอบ

มาจากไหน	FGI/FCS · migrate จาก ORA FGI_IMPACT_STORE_COMPENSATE (PK COMPENSATE_I_ID) · รับเข้าโครงเมื่อ 2026-08-21 (ปิดช่องว่าง F1)
ใช้ทำอะไร	เก็บ ยอดเงินชดเชยรายงวด ของรอบหนึ่ง ทั้งค่าที่ระบบคาดการณ์และค่าที่ STA ปรับกลับมา
ทำไมต้องมี	รอบชดเชยหนึ่งรอบกินเวลาหลายเดือน (เดือน 1–3 และอาจหยุดที่เดือน 4) จึงต้องมีแถวแยกต่อ งวด ไม่ใช่เก็บยอดเดียวที่แถวแม่ · ▲ ขากลับจาก STA ยังไม่มีตัวรับ — Job 11 ถูกตัดออก 2026-09-23 (มติผู้ใช้: job ที่ไม่มีโค้ดเดิมใน fcsJar) · adjust_amount จึงยังไม่มีผู้เขียน
ใครเขียน	Job 6 (ตั้งยอดคาดการณ์)
ใครอ่าน	Job 6 (ส่งไป STA) · รายงาน · sgi_compensation_histories

กติกาที่ต้องรู้ก่อนเขียนโค้ด

- compensate_status มีโดเมนปิด I/N/A/C/S/Z — C = ร้านปิดแล้ว หรือสัญญา SBP ถูกยกเลิก และตอนส่งไป STA ให้ส่งเป็น S (กลุ่มเดียวกับ Z)

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
id	BIGSERIAL	กุญแจแทน (surrogate key) ที่ฐานข้อมูลออกให้เอง เรียงเพิ่มทีละ 1	ใช้เป็น PK และเป็นเป้าของ FK จากตารางลูก · ห้ามเอาไปโชว์ผู้ใช้ (ผู้ใช้เห็น doc_no หรือรหัสร้านแทน) · เหตุผลที่ไม่ใช้ business key เป็น PK คือ business key ของระบบเดิมเปลี่ยนได้และบางตารางมีคีย์ผสม 3-4 คอลัมน์
impact_process_id	BIGINT	FK ไปรอบชดเชย	มัดยอดเข้ากับรอบ
impacted_store_code	VARCHAR(5)	รหัสร้านที่ได้รับชดเชย	เก็บซ้ำเพื่อ query รายร้านโดยไม่ต้อง join
compensate_seq	INTEGER	ลำดับรอบชดเชย	บอกว่าเป็นการชดเชยครั้งที่เท่าไรของร้านนี้
compensate_seq_no	INTEGER	ลำดับย่อยภายในรอบ	ใช้เมื่อรอบเดียวมีหลายงวดย่อย
compensate_month	CHAR(7)	งวดเดือนที่ชดเชย (YYYY-MM ค.ศ.)	ระบุว่ายอดก้อนนี้เป็นของเดือนไหน

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
compensate_year	INTEGER	ปี ค.ศ. ของงวด	แยกไว้เพื่อ query รายปี
forecast_amount	NUMERIC(14,2)	ยอดชดเชยที่ระบบคำนวณคาดการณ์ไว้	ค่าที่ SGI ส่งไปให้ STA ตั้งต้น
adjust_amount	NUMERIC(14,2)	ยอดหลัง STA ปรับแก้	▲ ยังไม่มีผู้เขียนในขอบเขตปัจจุบัน — คอลัมน์นี้มีไว้เพราะ migrate มาจาก ORA · job ที่เคยจะเขียน (Job 11) ถูกตัดออก 2026-09-23
compensate_status	VARCHAR(5)	สถานะของยอดก้อนนี้ (`I` ตั้งต้น · `N` ไม่ชดเชย · `A` อนุมัติ · `C` ร้านปิด/ยกเลิกสัญญา · `S/Z` กลุ่มส่งออก)	ตัวคุมว่ายอดนี้จะถูกส่งไป STA อย่างไร และจะนับเข้ารายงานหรือไม่
compensate_comment	VARCHAR(4000)	หมายเหตุประกอบยอด (ยาวได้ถึง 4000 ตัวอักษร)	เหตุผลที่ปรับยอดหรือไม่ชดเชย — ใช้ตอนตรวจสอบย้อนหลัง
stmt_month	INTEGER	เดือนของ statement ที่ยอดนี้ไปปรากฏ	ใช้กระทบยอดกับฝั่งบัญชี/Statement
stmt_year	INTEGER	ปีของ statement	คู่กับ `stmt_month`
approve_date	DATE	วันที่ยอดนี้ได้รับอนุมัติ	ใช้ในรายงานและการกระทบยอดกับบัญชี
created_by	VARCHAR(100)	ผู้สร้างแถว — `x-user-id` ของผู้ใช้จริง หรือชื่อ job (เช่น `JOB-8`) เมื่อ batch เป็นคนสร้าง	ใช้ตอบคำถาม "ใครทำให้เกิดแถวนี้" · แทนระบบ audit ที่ตัดทิ้งไปเมื่อ 2026-08-07 จึงเป็นร่องรอยชั้นเดียวที่เหลือ
created_at	TIMESTAMP	เวลาที่แถวนี้ถูกสร้าง (ตั้งอัตโนมัติด้วย `DEFAULT CURRENT_TIMESTAMP`)	ใช้เรียงลำดับเหตุการณ์ · เป็นฐานคำนวณอายุงานของ watchdog · ใช้สืบย้อนตอน incident ว่าแถวเกิดตอนไหน — ห้ามให้แอปเขียนทับ
updated_by	VARCHAR(100)	ผู้แก้ไขล่าสุด (รูปแบบเดียวกับ `created_by`)	ใช้ตอบคำถาม "ใครแก้ล่าสุด" ตอนข้อมูลเพี้ยน
updated_at	TIMESTAMP	เวลาที่แถวนี้ถูกแก้ล่าสุด	ใช้ตรวจว่าข้อมูล master ถูกแก้เมื่อไร และใช้เป็นตัวช่วยตอน sync/รีเฟรช cache ฝั่ง FE

sgi_interface_transactions

26 คอลัมน์

สมุดกลางของการรับ–ส่งข้อมูลกับระบบภายนอก (tracking + transactional outbox)

มาจากไหน	ตารางใหม่ · แทน ORA `FGI_CONFIRM_RECEIVE_DATA` ซึ่งมีบั๊ก 2 ข้อ: `TRANSACTION_PK` เป็น polymorphic FK และงาน purge ไม่ทำงานจริง
ใช้ทำอะไร	บันทึก ทุกครั้งที่ระบบรับหรือส่งข้อมูลกับระบบภายนอก (IAS · STA · การส่งต่อภายใน) 1 แถว = 1 record ที่รับส่ง · และทำหน้าที่เป็น transactional outbox ของข้อความ RabbitMQ
ทำไมต้องมี	สองเรื่องในตารางเดียว (ตั้งใจ): (1) ตามรอย ว่าอะไรส่งไปแล้ว/ยังไม่ถึง (2) กันข้อความหาย — Job 6 เขียนแถว `READY` ใน transaction เดียวกับการอัปเดตข้อมูล แล้วค่อย publish นอก transaction ถ้าระบบล่มกลางทาง แถว READY ยังอยู่และส่งซ้ำได้
ใครเขียน	Job 4/5/6/8/9 (เขียน outbox + tracking)
ใครอ่าน	Job 10 (watchdog ข้อความค้างส่ง) · ผู้ดูแลระบบผ่าน SQL

กติกาที่ต้องรู้ก่อนเขียนโค้ด

- UNIQUE (`data_name`, `direction`, `business_key`, `period_key`) — กันส่ง/รับซ้ำ และเป็นกลไก idempotent หลักของทุก job
- `ck_interface_typed_reference` บังคับว่าต้องมีอย่างน้อย 1 ใน `impact_process_id` / `sales_summary_id` / `doc_no` — แก้บั๊ก polymorphic FK ของระบบเดิม
- • มติ 2026-09-08 (ข้อ 2.13): ระบบ STA ไม่ตอบรับระดับธุรกิจกลับมา — ความสำเร็จของขาออกวัดจาก `outbox_status = 'CONFIRMED'` (publisher confirm ของ RabbitMQ)

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
id	BIGSERIAL	กุญแจแทน (surrogate key) ที่ฐานข้อมูลออกให้เอง เรียงเพิ่มทีละ 1	ใช้เป็น PK และเป็นเป้าของ FK จากตารางลูก · ห้ามเอาไปโชว์ผู้ใช้ (ผู้ใช้เห็น doc_no หรือรหัสร้านแทน) · เหตุผลที่ไม่ใช้ business key เป็น PK คือ business key ของระบบเดิมเปลี่ยนได้และบางตารางมีคีย์ผสม 3-4 คอลัมน์
run_id	VARCHAR(50)	รหัสรอบการรันที่สร้างแถวนี้ (มาจาก application log)	ใช้ผูกทุกแถวที่เกิดจากการรันครั้งเดียวกัน ตอนไล่หาสาเหตุ · ไม่มี FK เพราะตาราง `job_run_histories` ถูกตัดไปแล้ว
data_name	VARCHAR(80)	ชื่อชุดข้อมูลที่รับส่ง เช่น `COMPENSATE_INIT_I` · `IMPACT_STORE_SALES` (โดเมนบีด 12 ค่า)	เป็นส่วนหนึ่งของ UNIQUE ที่กันส่งซ้ำ และเป็นตัวกรองของ watchdog · พิมพ์ผิดตัวเดียว = กันซ้ำไม่ทำงาน + watchdog เงียบ จึงต้องล็อกด้วย CHECK
direction	VARCHAR(10)	ทิศทาง: `OUT` ส่งออก · `IN` รับเข้า · `INTERNAL` ส่งต่อภายในระบบ	`INTERNAL` คือสิ่งที่มาแทนไฟล์ EAI เดิม (Jobs 7/8/9 เขียน DB ตรง) จึงจบที่ `COMPLETED` ทันที ไม่มีอะไรให้รอ · watchdog นับเฉพาะ `OUT`
status	VARCHAR(20)	วงจรชีวิตของแถว: `READY` → `SENT` → `COMPLETED` (หรือ `FAILED`/`FAILED_RETRY`)	สถานะระดับ *แถว* · ตัดค่า ACKED ออกเมื่อ 2026-09-08 เพราะไม่มี ACK จาก STA ให้รอ
impact_process_id	BIGINT	typed FK ไปรอบชดเชย (ถ้าแถวนี้เกี่ยวกับรอบ)	หนึ่งใน 3 ทางอ้างอิงที่มาแทน polymorphic FK เดิม
sales_summary_id	BIGINT	typed FK ไปหัวยอดขาย (ถ้าแถวนี้เกี่ยวกับยอดขาย)	ใช้กับ dataset ของ Job 4/5
doc_no	VARCHAR(10)	typed FK ไปเอกสาร (ถ้าแถวนี้เกี่ยวกับเอกสาร)	nullable เพราะบางข้อความเกิดก่อนมีเลขเอกสาร · ผูกด้วย `impact_process_id` แทนได้
business_key	VARCHAR(200)	กุญแจธุรกิจของ record ที่รับส่ง เช่น รหัสร้าน หรือ `storecode_i:storecode_n`	ส่วนหนึ่งของ UNIQUE ที่กันซ้ำ — เป็นตัวบอกว่า "ข้อความนี้พูดถึงของชิ้นไหน"

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
period_key	VARCHAR(20)	งวดของ record	อีกส่วนของ UNIQUE — ของชิ้นเดียวกันคนละงวดถือเป็นคนละข้อความ
correlation_id	VARCHAR(100)	รหัสสำหรับจับคู่ข้อความขาไป–ขากลับ	▲ job ต้องอ่านค่านี้จาก envelope เอง (มติ 2026-09-12 · job consume คิวเอง) — ถ้าต้นทางไม่ส่งมาต้องยืนยันกับเจ้าของข้อความ
file_name	VARCHAR(255)	ชื่อไฟล์ต้นทาง/ปลายทาง (กรณี interface แบบไฟล์)	ใช้ตามรอยว่าข้อมูลมาจากไฟล์ไหนบน EAI S3
file_checksum	VARCHAR(64)	checksum ของไฟล์	พิสูจน์ว่าไฟล์ที่ประมวลผลคือไฟล์เดียวกับที่ต้นทางส่ง และกันประมวลผลไฟล์ซ้ำ
outbox_status	VARCHAR(20)	สถานะการ publish: `READY` รอส่ง · `PUBLISHED` ยิงแล้วรอ confirm · `CONFIRMED` broker ยืนยันแล้ว · `FAILED` ส่งไม่สำเร็จ	● เกณฑ์ตัดสินของ Job 10 ตั้งแต่มติ 2026-09-08 — แถว `OUT` ที่ไม่ใช่ `CONFIRMED` เกิน 1 วัน = ค้างส่ง ต้องส่งอีเมลเตือน
return_code	VARCHAR(50)	รหัสผลลัพธ์ที่ปลายทางตอบกลับ (ถ้ามี)	ใช้กับ interface ที่มีการตอบกลับ — ไม่ใช่เกณฑ์ของ watchdog แล้ว
return_message	VARCHAR(500)	ข้อความผลลัพธ์จากปลายทาง	ใช้อ่านตอนแก้ปัญหา
payload	JSONB	ข้อความที่ส่งออกไปจริง (JSONB)	เก็บของจริงไว้เพื่อส่งซ้ำได้ตรงตัว — ถ้าประกอบใหม่จากข้อมูลในฐานที่เปลี่ยนไปแล้ว ปลายทางจะได้คนละฉบับกับที่เคยส่งโดยไม่มีใครรู้ · เก็บเฉพาะขาออก
payload_version	SMALLINT	เวอร์ชันรูปแบบของ payload	ขึ้นเมื่อ contract กับปลายทางเปลี่ยน ไม่ใช่เมื่อข้อมูลเปลี่ยน — ใช้ตอนอ่าน payload เก่ากลับมาส่งซ้ำ
retry_count	INTEGER	จำนวนครั้งที่ลองส่งซ้ำ	ใช้ตัดสินว่าควรหยุดลองแล้วแจ้งคนหรือยัง
sent_at	TIMESTAMP	เวลาที่ publish ออกไป	ใช้วัดว่าค้างนานแค่ไหนหลังส่ง
acked_at	TIMESTAMP	เวลาที่ได้ publisher confirm จาก broker	ชื่อคอลัมน์คงเดิมเพื่อ compatibility แต่ความหมายเปลี่ยนตามมติ 2026-09-08 — ไม่ใช่เวลาที่ STA ตอบรับ
last_ack_notified_on	DATE	วันที่ watchdog ส่งอีเมลเตือนแถวนี้ล่าสุด	marker กันส่งอีเมลซ้ำในวันเดียวกัน ตอน Job 10 รันซ้ำ · ย้ายมาจากตาราง `audit_logs` ที่ถูกตัดเมื่อ 2026-08-07
purge_after	TIMESTAMP	วันที่แถวนี้ลบทิ้งได้	ใช้โดย retention worker — ลบเฉพาะแถว `COMPLETED` ที่เลยกำหนดและไม่ติด legal hold
legal_hold	BOOLEAN	ธงห้ามลบ (มีคดี/ตรวจสอบ)	กัน retention worker ลบหลักฐานที่ยังต้องใช้
created_at	TIMESTAMP	เวลาที่แถวนี้ถูกสร้าง (ตั้งอัตโนมัติด้วย `DEFAULT CURRENT_TIMESTAMP`)	ใช้เรียงลำดับเหตุการณ์ · เป็นฐานคำนวณอายุงานของ watchdog · ใช้สืบย้อนตอน incident ว่าแถวเกิดตอนไหน — ห้ามให้แอปเขียนทับ
completed_at	TIMESTAMP	เวลาที่แถวนี้จบวงจร	ใช้วัด lead time ของ interface และเป็นฐานของ `purge_after`

โซน B เอกสารประกันรายได้ (workflow ใช้ engine กลาง) — 9 ตาราง

sgi_compensation_documents

31 คอลัมน์

เอกสารประกันรายได้ (หัวเอกสาร) — ตารางกลางของทั้งโซน B

มาจากไหน	K2 · migrate จาก MSSQL `CompensateFlow` (PK `CompDocumentID` → `doc_no`) join กับ ORA `FGI_IMPACT_STORE_INFO` เพื่อเติม `impact_process_id`
ใช้ทำอะไร	เก็บ เอกสารขอชดเชยหนึ่งฉบับ — สิ่งที่ผู้ใช้เห็น อนุมัติ และส่งต่อใน workflow 5 ขั้น
ทำไมต้องมี	เป็นจุดเชื่อมระหว่างโลก batch (โซน A) กับโลกคน (workflow อนุมัติ) · ทุกตารางลูกในโซน B ห้อยจาก doc_no ของตารางนี้ และ engine workflow อ้างเอกสารนี้ผ่าน `referenceId`
ใครเขียน	Job 8 (CreateCompensationDocument) · `POST/PUT /sgi/document` · `POST /sgi/document/{docNo}/actions`
ใครอ่าน	ทุกหน้าจอฝั่ง FE · รายงาน SBP Mall · Job 6 (ส่งยอดไป STA)

กติกาที่ต้องรู้ก่อนเขียนโค้ด

- ✓ มติ DP-1 (2026-08-10): PK เป็น surrogate `id` · `doc_no` เป็น UNIQUE ไม่ใช่ PK — `referenceId` ที่ส่งให้ `@srm/glb-workflow` คือ `id` (ส่งเป็น string เพราะ `reference_id` เป็น varchar(255))
- ▲ ตารางลูก 6 ตัวมี `doc_no` เป็น NOT NULL จึงต้องออกเลขเอกสารใน INSERT เดียวกับที่สร้างหัวเสมอ
- ไม่มีคอลัมน์ `assignee` — ผู้รับผิดชอบปัจจุบันอยู่ที่ engine (`workflow_transaction.current_approver`) ที่เดียว

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
id	BIGSERIAL	กุญแจแทน (surrogate key) ที่ฐานข้อมูลออกให้เอง เรียงเพิ่มทีละ 1	ใช้เป็น PK และเป็นเป้าของ FK จากตารางลูก · ห้ามเอาไปโชว์ผู้ใช้ (ผู้ใช้เห็น doc_no หรือรหัสร้านแทน) · เหตุผลที่ไม่ใช้ business key เป็น PK คือ business key ของระบบเดิมเปลี่ยนได้และบางตารางมีคีย์ผสม 3-4 คอลัมน์
doc_no	VARCHAR(10)	เลขที่เอกสาร รูปแบบ YYYY/xxxxx โดย YYYY เป็น ค.ศ.	เลขที่ผู้ใช้เห็นและใช้อ้างอิงทุกที่ · เป็น UNIQUE และเป็นเป้าของ FK จากตารางลูก 8 ตัว · ▲ ต้อง `encodeURIComponent` เสมอเมื่อใส่ใน URL เพราะมี `/`
year	INTEGER	ปี ค.ศ. ของเอกสาร	คู่กับ `running_no` เพื่อประกอบเป็น `doc_no` และใช้กรองรายปี
running_no	INTEGER	เลขลำดับในปีนั้น	ได้มาจาก `sgi_document_running_numbers` — เลขนี้ห้ามซ้ำในปีเดียวกัน
impact_process_id	BIGINT	FK ย้อนกลับไปรอบชดเชยในโซน A	เส้นเชื่อมสองโลก — ทำให้จากเอกสารย้อนไปดูยอดขาย/ร้านเปลี่ยนใหม่/คู่แข่งที่เป็นต้นเหตุได้
impact_compensation_id	BIGINT	แถวค่าชดเชยของงวดนั้น ที่เอกสารใบนี้ว่าด้วย (1:1)	ตัวตนจริงของเอกสาร — รอบชดเชยหนึ่งรอบกินหลายงวด และแต่ละงวดมีเอกสารของตัวเอง · ORA `FGI_IMPACT_STORE_INFO` 20,676 แถว ต่อค่าชดเชยรายงวด 20,674 แถว = 1.0001 · ใช้เป็นคีย์กันสร้างเอกสารซ้ำ (`ON CONFLICT`) แทน `impact_process_id` ที่เดิมผิด
impacted_store_code	VARCHAR(5)	รหัสร้านที่ขอชดเชย	ใช้แสดงหัวเอกสารและกรองรายงานโดยไม่ต้อง join ขึ้นโซน A
impact_month	CHAR(7)	งวดเดือนที่ขอชดเชย (YYYY-MM ค.ศ.)	คู่กับรหัสร้านเป็น business key ของเอกสาร
new_store_code	VARCHAR(5)	รหัสร้านเปิดใหม่หลักของเอกสารนี้	ใช้แสดงหัวเอกสาร — รายละเอียดครบอยู่ที่ `sgi_document_new_stores`
round_no	INTEGER	รอบที่เท่าไรของการยื่นเรื่องนี้	รองรับกรณี เปิดเอกสารใหม่ได้เองหลังจบด้วยหยุด/ไม่เห็นควรชดเชย (SDD GI ตัดค่าเปิด SR เดือนละ ~20,000 บาท)

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
loop_no	INTEGER	รอบการวนกลับภายใน workflow	นับจำนวนครั้งที่เอกสารถูกส่งกลับมาแก้ — ใช้ดูว่าเอกสารวนนานผิดปกติไหม
source	VARCHAR(20)	ที่มาของเอกสาร: FS (สร้างจาก pipeline) หรือ MANUAL (คีย์มือ)	ใช้แยกว่าเอกสารมาจากงานอัตโนมัติหรือคน — มีผลต่อการตรวจสอบเมื่อข้อมูลไม่ตรง
status_code	VARCHAR(2)	รหัสสถานะเอกสาร (1 ใน 6 ค่า)	ตัวขับสีป้ายสถานะและตัวกรองหลักของหน้ารายการ/รายงาน · ค่าที่เป็นไปได้ตรงกับ `workflow_status_document.md`
current_section_code	VARCHAR(2)	รหัสขั้นที่เอกสารค้างอยู่ (06/08/01/02/03/99)	ใช้ตอบว่า "ตอนนี้ใครต้องทำ" · เป็นสำเนาไว้ query เร็ว — ค่าจริงที่ตัดสินสิทธิ์คือของ engine ผ่าน `getTransaction()`
total_compensation_amount	NUMERIC(14,2)	ยอดชดเชยรวมของเอกสาร (บาท)	ตัวตัดสินเส้นทางอนุมัติ: < 100,000 จบที่ GM (02) · ≥ 100,000 ไปต่อ AVP (03) (เกณฑ์เดียว มติ 2026-08-18)
allmap_url	VARCHAR(500)	ลิงก์แผนที่ ALLMAP ของร้านนี้	ปุ่มเปิดแผนที่ในหน้าเอกสาร — เก็บ URL เพราะแผนที่เป็นระบบทีมอื่น
statement_id	VARCHAR(500)	เลขอ้างอิง statement ฝั่งบัญชี	ใช้กระทบยอดกับระบบ Statement/STA
statement_date	DATE	วันที่ของ statement	คู่กับ `statement_id`
account_year	INTEGER	ปีบัญชีที่ยอดนี้ลง	ใช้ในรายงานฝั่งบัญชี
account_month	INTEGER	เดือนบัญชีที่ยอดนี้ลง	คู่กับ `account_year` — บัญชีใช้ค้นด้วยงวดบัญชี ไม่ใช่งวดกระทบ
approver_snapshot	JSONB	สำเนา ชื่อและตำแหน่งผู้อนุมัติ ณ เวลาที่อนุมัติ (JSONB)	เก็บเป็น snapshot เพราะคนย้ายตำแหน่ง/ลาออกได้ — เอกสารเก่าต้องยังพิมพ์ชื่อผู้อนุมัติเดิมได้ถูก · ▲ เป็น ข้อมูลส่วนบุคคล อยู่ในขอบเขต PDPA (ข้อค้าง 2.6)
draft_result	VARCHAR(100)	ร่างผลการพิจารณาที่ผู้ใช้เลือกไว้แล้วกด "บันทึก" แต่ยังไม่ส่งดำเนินการ	เพิ่ม 2026-10-07 (มติผู้ใช้) — เปิดเอกสารใหม่แล้ว radio ผลพิจารณาต้องเป็นค่าเดิม · ต้องเป็นตัวเลือกของขั้นปัจจุบัน · ล้างเป็น NULL เมื่อส่งดำเนินการสำเร็จ
draft_comment	TEXT	ร่างความเห็น/รายละเอียดที่พิมพ์ไว้คู่กับ `draft_result`	ไม่ใช่ประวัติ — ความเห็นที่ส่งจริงอยู่ที่ `sgi_consideration_logs.detail`
draft_section_code	VARCHAR(2)	ขั้นของเอกสารตอนที่บันทึกร่าง	หน้ารายละเอียดคืนร่างเฉพาะเมื่อยังเท่ากับ `current_section_code` (ขั้นเปลี่ยนแล้ว ตัวเลือกของขั้นเดิมใช้ไม่ได้)
draft_by	VARCHAR(30)	รหัสผู้ใช้ที่บันทึกร่างล่าสุด (`sps_auth.users.id`)	คนเดียวกับเจ้าของคิวของขั้นนั้น
draft_at	TIMESTAMP	เวลาที่บันทึกร่างล่าสุด (เวลาไทย)	เขียนด้วยเวลาไทยแบบเดียวกับคอลัมน์เวลาอื่นของ SGI
version_no	INTEGER	เลขเวอร์ชันของแถว เพิ่มทีละ 1 ทุกครั้งที่แก้	optimistic locking — ถ้าสองคนเปิดเอกสารเดียวกันแล้วกดบันทึกพร้อมกัน คนที่สองจะได้ `409 STALE_VERSION` แทนที่จะเขียนทับเงียบ ๆ · บันทึกร่างผลพิจารณาอย่างเดียวไม่เลื่อนเลขนี้
created_by	VARCHAR(30)	ผู้สร้างแถว — `x-user-id` ของผู้ใช้จริง หรือชื่อ job (เช่น `JOB-8`) เมื่อ batch เป็นคนสร้าง	ใช้ตอบคำถาม "ใครทำให้เกิดแถวนี้" · แทนระบบ audit ที่ตัดทิ้งไปเมื่อ 2026-08-07 จึงเป็นร่องรอยชั้นเดียวที่เหลือ
created_at	TIMESTAMP	เวลาที่แถวนี้ถูกสร้าง (ตั้งอัตโนมัติด้วย `DEFAULT CURRENT_TIMESTAMP`)	ใช้เรียงลำดับเหตุการณ์ · เป็นฐานคำนวณอายุงานของ watchdog · ใช้สืบย้อนตอน incident ว่าแถวเกิดตอนไหน — ห้ามให้แอปเขียนทับ
updated_by	VARCHAR(30)	ผู้แก้ไขล่าสุด (รูปแบบเดียวกับ `created_by`)	ใช้ตอบคำถาม "ใครแก้ล่าสุด" ตอนข้อมูลเพี้ยน
updated_at	TIMESTAMP	เวลาที่แถวนี้ถูกแก้ล่าสุด	ใช้ตรวจว่าข้อมูล master ถูกแก้เมื่อไร และใช้เป็นตัวช่วยตอน sync/รีเฟรช cache ฝั่ง FE

sgi_document_new_stores

9 คอลัมน์

ร้านเปิดใหม่ในเอกสาร + การปันส่วน %ชดเชย

มาจากไหน	K2 · migrate จาก MSSQL `ImpactProfile` (ฝั่ง _N) + ORA `FGI_NEW_STORE_INFO` / `FGI_NEW_STORE_COMPENSATE`
ใช้ทำอะไร	เก็บว่าเอกสารฉบับนี้ขอชดเชยจาก ร้านเปิดใหม่ร้านไหนบ้าง ร้านละกี่ % และเป็นเงินเท่าไร
ทำไมต้องมี	เป็นตารางที่ผู้ใช้ แก้ได้จริงในหน้าจอ (ปรับ %) ต่างจาก `sgi_fgi_impact_stores` ที่เป็นข้อมูล pipeline · การแยกสองตารางทำให้แก้ในเอกสารแล้วไม่ไปทับข้อมูลดิบของ batch
ใครเขียน	Job 9 (sync จากโซน A) · หน้าเอกสารผ่าน `PUT /sgi/document/{docNo}`
ใครอ่าน	หน้าเอกสาร · การคำนวณ `total_compensation_amount` · Job 6

กติกาที่ต้องรู้ก่อนเขียนโค้ด

- ผลรวม `compensate_percent` ของทุกแถวใน `doc_no` เดียวกันต้องเท่ากับ 100 พอดี — validate ก่อน commit

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
id	BIGSERIAL	กุญแจแทน (surrogate key) ที่ฐานข้อมูลออกให้เอง เรียงเพิ่มทีละ 1	ใช้เป็น PK และเป็นเป้าของ FK จากตารางลูก · ห้ามเอาไปโชว์ผู้ใช้ (ผู้ใช้เห็น doc_no หรือรหัสร้านแทน) · เหตุผลที่ไม่ใช้ business key เป็น PK คือ business key ของระบบเดิมเปลี่ยนได้และบางตารางมีคีย์ผสม 3-4 คอลัมน์
doc_no	VARCHAR(10)	FK ไปหัวเอกสาร (NOT NULL)	มัดแถวเข้ากับเอกสาร · `ON DELETE CASCADE` — ลบเอกสารแล้วแถวนี้หายตาม
new_store_code	VARCHAR(5)	รหัสร้านเปิดใหม่	join ไป master ร้านของระบบเดิมเพื่อเอาชื่อ/ที่อยู่
distance_km	NUMERIC(8,3)	ระยะทางจากร้านเปิดใหม่ถึงร้าน SP	แสดงในตารางให้ผู้อนุมัติเห็นว่าใกล้แค่ไหน · ต้องอยู่ในรัศมีเกณฑ์
compensate_percent	NUMERIC(7,4)	%ชดเชยที่ปันให้ร้านนี้	ตัวคูณคำนวณ `compensation_amount` · รวมทุกแถวต้อง = 100
compensation_amount	NUMERIC(14,2)	ยอดเงินชดเชยของร้านนี้	= ยอดรวมเอกสาร × `compensate_percent` · รวมทุกแถว = `total_compensation_amount`
source_system	VARCHAR(30)	ที่มาของแถว: `FGI` (มาจาก pipeline) หรือ `USER` (ผู้ใช้เพิ่มเอง)	สำคัญมาก — ตอน Job 9 sync ใหม่จะ prune เฉพาะแถว `FGI` เท่านั้น แถวที่ผู้ใช้เพิ่มเองห้ามลบ
source_row_id	BIGINT	แถวต้นทางใน `sgi_fgi_new_store_compensations` ที่ Job 9 คัดลอกมา	%ชดเชยผิด = จ่ายเงินผิด จึงต้องตามรอยกลับได้ว่าตัวเลขมาจากแถวไหนของงวดไหน · NULL = แถวที่ผู้ใช้เพิ่มเอง
updated_at	TIMESTAMP	เวลาที่แถวนี้ถูกแก้ล่าสุด	ใช้ตรวจว่าข้อมูล master ถูกแก้เมื่อไร และใช้เป็นตัวช่วยตอน sync/รีเฟรช cache ฝั่ง FE

sgi_document_competitors

18 คอลัมน์

ร้านคู่แข่งในเอกสาร (ระดับสาขา)

มาจากไหน	K2 · migrate จาก MSSQL `CompetInCompenProfile` (+ ไฟล์ `BPM06003O_` 14 ฟิลด์)
ใช้ทำอะไร	เก็บ สาขาคู่แข่งที่ยกมาประกอบเอกสารฉบับนี้ พร้อมรายละเอียดและหมายเหตุที่ผู้ใช้เขียนเอง
ทำไมต้องมี	ผู้พิจารณาต้องเห็นว่ามีคู่แข่งเปิดใหม่ในพื้นที่ด้วยหรือไม่ · และต้อง เพิ่ม/แก้ได้เอง เพราะ ALLMAP ไม่ได้ครบเสมอ — จึงคัดลอกมาเป็นแถวของเอกสารแทนการอ่านจาก pipeline ตรง ๆ
ใครเขียน	Job 7 (sync จาก `sgi_fgi_impact_competitors`) · หน้าเอกสาร
ใครอ่าน	หน้าเอกสาร · รายงาน

กติกาที่ต้องรู้ก่อนเขียนโค้ด

- ▲ คนละตารางกับ `sgi_competitors` — ที่นี่คือสาขา (หลายแถวต่อเอกสาร) ส่วนอีกตารางคือแบรนด์ (11 แถวทั้งระบบ)

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
id	BIGSERIAL	กุญแจแทน (surrogate key) ที่ฐานข้อมูลออกให้เอง เรียงเพิ่มทีละ 1	ใช้เป็น PK และเป็นเป้าของ FK จากตารางลูก · ห้ามเอาไปโชว์ผู้ใช้ (ผู้ใช้เห็น doc_no หรือรหัสร้านแทน) · เหตุผลที่ไม่ใช้ business key เป็น PK คือ business key ของระบบเดิมเปลี่ยนได้และบางตารางมีคีย์ผสม 3-4 คอลัมน์
doc_no	VARCHAR(10)	FK ไปหัวเอกสาร (NOT NULL · CASCADE)	มัดแถวเข้ากับเอกสาร
competitor_store_code	VARCHAR(50)	รหัสสาขาของคู่แข่ง (คัดลอกมาจาก Job 7)	ไม่มี FK · nullable เพราะแถวที่ผู้ใช้คีย์เองอาจไม่มีรหัสสาขา · เป็นคีย์กันซ้ำต่อเอกสาร
brand_code	VARCHAR(30)	รหัสแบรนด์ `01–11`	FK ไป `sgi_competitors` — ทำให้สรุปรายงานรายแบรนด์ได้ · nullable เมื่อ map ไม่ได้
name_th	VARCHAR(200)	ชื่อร้านคู่แข่งภาษาไทย	แสดงในตารางของหน้าเอกสาร
name_en	VARCHAR(200)	ชื่อแบรนด์ภาษาอังกฤษ	ส่งต่อ K2 ในไฟล์ BPM06003O
branch_th	VARCHAR(200)	ชื่อสาขา	ระบุสาขาที่กระทบ
zone_code	VARCHAR(10)	โซนของสาขาคู่แข่ง	ส่งต่อ K2
subzone_code	VARCHAR(10)	โซนย่อยของสาขาคู่แข่ง	ส่งต่อ K2
opened_date	DATE	วันที่สาขานี้เปิด	ใช้เทียบไทม์ไลน์กับวันที่ร้าน 7-Eleven ใหม่เปิด
closed_date	DATE	วันที่สาขานี้ปิด (ถ้ามี)	สาขาที่ปิดแล้วไม่ควรถูกนับเป็นเหตุกระทบของงวดถัดมา
impact_date	DATE	วันที่เริ่มกระทบตามที่ผู้ใช้ระบุ	ผู้ใช้ระบุเองได้เมื่อวันเปิดไม่ตรงกับวันที่เริ่มกระทบจริง
detail	TEXT	รายละเอียดที่ผู้ใช้พิมพ์	คำอธิบายประกอบการพิจารณา
remark	TEXT	หมายเหตุเพิ่มเติม	บันทึกช่วยจำของผู้ทำเรื่อง

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
source_system	VARCHAR(30)	ALLMAP (มาจาก pipeline) หรือ USER (ผู้ใช้เพิ่มเอง)	ตัวคุมว่าตอน Job 7 sync ใหม่จะ prune แถวไหนได้ — แถว USER ห้ามลบ
source_row_id	BIGINT	แถวต้นทางใน sgi_fgi_impact_competitors ที่ Job 7 คัดลอกมา	ตามรอยย้อนกลับตอน audit — ผู้พิจารณาทักว่าข้อมูลคู่แข่งผิด ต้องรู้ว่ามาจากแถวไหน · ระบบเดิมส่ง PK ต้นทางเป็นฟิลด์ที่ 1 ของไฟล์ BPM06003O · NULL = แถวที่ผู้ใช้คีย์เอง (source_system = 'USER') ซึ่งไม่มีต้นทาง
updated_at	TIMESTAMP	เวลาที่แถวนี้ถูกแก้ล่าสุด	ใช้ตรวจว่าข้อมูล master ถูกแก้เมื่อไร และใช้เป็นตัวช่วยตอน sync/รีเฟรช cache ฝั่ง FE
competitor_key	VARCHAR(300)	คีย์ธุรกิจของสาขาคู่แข่ง — generated: ใช้ competitor_store_code ถ้ามี ไม่งั้น fallback เป็น name:<ชื่อสาขา>	ALLMAP ส่งสาขาที่ไม่มีรหัสมาได้ (ORA COMPET_ID ไม่มี NOT NULL) · ใช้รหัสอย่างเดียวเป็นคีย์ไม่ได้: NULLS DISTINCT ทำให้แถวพอกทุกรอบ · NULLS NOT DISTINCT ยุบหลายสาขาเหลือแถวเดียว = ข้อมูลหาย (ร้าน 07109 มีแฟมิลี่มาร์ท 80 สาขา) · คอลัมน์นี้จึงแยกสาขาด้วยชื่อเมื่อไม่มีรหัส

sgi_document_external_factors

8 คอลัมน์

ปัจจัยภายนอกที่ยกมาในเอกสาร

มาจากไหน	K2 · migrate จาก MSSQL FactorInCompenProfile
ใช้ทำอะไร	เก็บว่าเอกสารฉบับนี้อ้าง ปัจจัยภายนอกข้อไหน ช่วงวันที่เท่าไร เช่น ปิดถนนตั้งแต่วันไหนถึงวันไหน
ทำไมต้องมี	ปัจจัยภายนอกเป็นเหตุผลที่ทำให้ยอดตกโดยไม่เกี่ยวกับร้านเปิดใหม่ · ต้องมีช่วงวันที่เพื่อให้ผู้อนุมัติตัดสินว่าคาบเกี่ยวกับงวดที่ขอชดเชยจริงไหม
ใครเขียน	หน้าเอกสารผ่าน PUT /sgi/document/{docNo}
ใครอ่าน	หน้าเอกสาร · รายงาน

กติกาที่ต้องรู้ก่อนเขียนโค้ด

- date_to เป็น optional — ปัจจัยที่ยังไม่จบ (เช่น ก่อสร้างยังไม่เสร็จ) ปล่อยว่างได้

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
id	BIGSERIAL	กุญแจแทน (surrogate key) ที่ฐานข้อมูลออกให้เอง เรียงเพิ่มทีละ 1	ใช้เป็น PK และเป็นเป้าของ FK จากตารางลูก · ห้ามเอาไปโชว์ผู้ใช้ (ผู้ใช้เห็น doc_no หรือรหัสร้านแทน) · เหตุผลที่ไม่ใช้ business key เป็น PK คือ business key ของระบบเดิมเปลี่ยนได้และบางตารางมีคีย์ผสม 3-4 คอลัมน์
doc_no	VARCHAR(10)	FK ไปหัวเอกสาร (NOT NULL · CASCADE)	มัดแถวเข้ากับเอกสาร
factor_code	VARCHAR(30)	รหัสปัจจัยที่เลือกจาก master	FK ไป sgi_external_factors — บังคับให้เลือกจากรายการ ไม่ให้พิมพ์อิสระ
date_from	DATE	วันที่ปัจจัยเริ่มมีผล	ใช้ตรวจว่าคาบเกี่ยวกับงวดที่ขอชดเชยหรือไม่
date_to	DATE	วันที่ปัจจัยสิ้นสุด (ว่างได้ = ยังไม่จบ)	คู่กับ date_from เป็นช่วงเวลา

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
detail	TEXT	รายละเอียดของปัจจัยในบริบทของเอกสารนี้	อธิบายว่ากระทบร้านนี้อย่างไร
remark	TEXT	หมายเหตุเพิ่มเติม	บันทึกช่วยจำ
updated_at	TIMESTAMP	เวลาที่แถวนี้ถูกแก้ล่าสุด	ใช้ตรวจว่าข้อมูล master ถูกแก้เมื่อไร และใช้เป็นตัวช่วยตอน sync/รีเฟรช cache ฝั่ง FE

sgi_consideration_logs

10 คอลัมน์

ประวัติการพิจารณาทุกขั้น (ใครตัดสินอะไร เมื่อไร)

มาจากไหน	K2 · migrate จาก MSSQL `CompensateHistory` (PK `ActionID`)
ใช้ทำอะไร	บันทึก ทุกครั้งที่มีคนกดปุ่มตัดสินใจ ในเอกสาร — ขั้นไหน ผลอะไร ใครทำ เมื่อไร พร้อมความเห็น
ทำไมต้องมี	engine `@srm/glb-workflow` เก็บประวัติของตัวเองก็จริง แต่ ไม่มีที่เก็บรหัสผลการพิจารณา และความเห็นแบบที่ธุรกิจนี้ต้องการ — จึงเป็นหนึ่งใน 3 ตารางที่เก็บไว้เองแม้จะดูซ้ำกับ engine
ใครเขียน	`POST /sgi/document/{docNo}/actions` — เขียนใน transaction เดียวกับการ trigger event ของ engine
ใครอ่าน	ไทม์ไลน์ในหน้าเอกสาร · รายงานตรวจสอบ

กติกาที่ต้องรู้ก่อนเขียนโค้ด

- append-only — ห้าม UPDATE/DELETE แถวเก่า ประวัติต้องเรียงตามจริง
- ต้องเขียนใน transaction เดียวกับการเปลี่ยนสถานะเอกสาร ไม่งั้นประวัติกับสถานะจะไม่ตรงกัน

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
id	BIGSERIAL	กุญแจแทน (surrogate key) ที่ฐานข้อมูลออกให้เอง เรียงเพิ่มทีละ 1	ใช้เป็น PK และเป็นเป้าของ FK จากตารางลูก · ห้ามเอาไปโชว์ผู้ใช้ (ผู้ใช้เห็น doc_no หรือรหัสร้านแทน) · เหตุผลที่ไม่ใช้ business key เป็น PK คือ business key ของระบบเดิมเปลี่ยนได้และบางตารางมีคีย์ผสม 3-4 คอลัมน์
doc_no	VARCHAR(10)	FK ไปหัวเอกสาร (NOT NULL)	มัดประวัติเข้ากับเอกสาร · ไม่ CASCADE — ประวัติต้องอยู่แม้เอกสารถูกลบ
section_code	VARCHAR(2)	ขั้นที่เกิดการตัดสิน (`06/08/01/02/03`)	บอกว่าใครในสายอนุมัติเป็นคนตัดสิน
result	VARCHAR(100)	ผลการพิจารณาเป็นข้อความไทยตามที่ผู้ใช้เห็น (7 ค่า)	แสดงบนไทม์ไลน์ตรงตัว · ข้อความ verbatim จาก SRS ห้ามแก้คำ · ค่าที่ 7 คือ "คำนวณเงินชดเชยเรียบร้อย" เพิ่มเมื่อ 2026-09-01
result_category	VARCHAR(50)	หมวดของผล: `APPROVE` · `REJECT` · `PENDING` (ถอด `CANCELLED` 2026-10-12 · หยุดชดเชยนับเป็น REJECT)	ใช้จัดกลุ่มในรายงานโดยไม่ต้อง match ข้อความไทย ซึ่งเปลี่ยนคำได้
detail	TEXT	ความเห็น/เหตุผลที่ผู้พิจารณาพิมพ์	หลักฐานประกอบการตัดสิน — ใช้ตอนตรวจสอบย้อนหลัง
consider_by	VARCHAR(30)	ผู้พิจารณา (`x-user-id`)	ตอบว่าใครเป็นคนตัดสิน · ▲ เป็นข้อมูลส่วนบุคคลตาม PDPA

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
consider_by_name	VARCHAR(200)	ชื่อผู้พิจารณา ณ ตอนกด (จาก `x-user-full-name` = ชื่อใน sps_auth.users)	หน้าจอ/รายงานแสดงชื่อจากคอลัมน์นี้ · เพิ่ม 2026-10-05 · ▲ ข้อมูลส่วนบุคคลตาม PDPA
action_datetime	TIMESTAMP	เวลาที่ตัดสิน	เรียงไทม์ไลน์ และใช้คำนวณว่างานค้างที่ขั้นไหนนานเท่าไร (Job 12)
request_id	VARCHAR(80)	รหัสคำขอของ action นี้	idempotency key — ถ้าผู้ใช้กดปุ่มซ้ำหรือ retry เข้ามาด้วย `request_id` เดิม ต้องไม่เกิดประวัติซ้ำและไม่เลื่อน workflow สองครั้ง

sgi_document_attachments

14 คอลัมน์

ไฟล์แนบของเอกสาร (เก็บเฉพาะ metadata ตัวไฟล์อยู่บน S3)

มาจากไหน	K2 · migrate จาก MSSQL `CompDocAttachment` + `CompTempAttachment` (+ `AttachFileProfile`)
ใช้ทำอะไร	เก็บ ข้อมูลของไฟล์แนบ — ชื่อ ขนาด ชนิด ที่อยู่บน S3 และผลสแกนไวรัส
ทำไมต้องมี	ตัวไฟล์จริงอยู่บน S3 ผ่านบริการของระบบ SBP เดิม (`POST /statement/upload-file-aws`) · แต่ต้องมีตารางฝั่งเราเพื่อผูกไฟล์กับเอกสารและ ขั้นตอนที่แนบ ซึ่ง S3 ไม่รู้
ใครเขียน	`POST /sgi/document/{docNo}/attachments`
ใครอ่าน	หน้าเอกสาร · `GET .../download` และ `/download-all`

กติกาที่ต้องรู้ก่อนเขียนโค้ด

- ไฟล์ละไม่เกิน 5 MB (บังคับด้วย `CHECK (file_size <= 5242880)`)
- ไม่มีการสแกนไวรัส — ถอดคอลัมน์สถานะสแกนและสวิตช์ใน `mas_param` ออก 2026-10-05 (มติผู้ใช้ · ไม่เคยมีตัวสแกนจริง)
- ทุก query ต้องกรอง `deleted_flag = 'N'` เสมอ

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
attach_id	BIGSERIAL	PK ของไฟล์แนบ	ใช้ใน URL ดาวน์โหลดรายไฟล์
doc_no	VARCHAR(10)	FK ไปหัวเอกสาร (NOT NULL)	มัดไฟล์เข้ากับเอกสาร · ไม่ CASCADE เพราะไฟล์เป็นหลักฐาน
section_code	VARCHAR(2)	ขั้นที่ไฟล์นี้ถูกแนบ	ทำให้รู้ว่าไฟล์มาจากขั้นไหน และคุมสิทธิ์ว่าใครแนบ/ลบได้
file_name	VARCHAR(255)	ชื่อไฟล์ตามที่ผู้ใช้อัปโหลด	แสดงในตารางและใช้เป็นชื่อไฟล์ตอนดาวน์โหลด
mime_type	VARCHAR(100)	ชนิดไฟล์ เช่น `application/pdf`	ใช้ตั้ง `Content-Type` ตอนส่งไฟล์กลับ และกรองชนิดที่ไม่อนุญาต (415)
file_size	BIGINT	ขนาดไฟล์เป็นไบต์	บังคับเพดาน 5 MB ที่ระดับฐานข้อมูล — กันไฟล์ใหญ่หลุดเข้ามาแม้แอปพลาด
storage_provider	VARCHAR(30)	ผู้ให้บริการที่เก็บไฟล์ (เช่น S3)	เผื่อย้ายที่เก็บในอนาคตโดยไม่ต้องแก้โครง
bucket	VARCHAR(120)	ชื่อ bucket ที่เก็บไฟล์	ครึ่งหนึ่งของที่อยู่ไฟล์

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
object_key	VARCHAR(500)	path ของไฟล์ใน bucket	อีกครึ่ง — ใช้ประกอบกับ bucket เพื่อดึงไฟล์จริง
sha256	VARCHAR(64)	ลายนิ้วมือของไฟล์	ใช้กันอัปโหลดไฟล์เดิมซ้ำ (อยู่ใน UNIQUE `doc_no + sha256 + deleted_flag`) และพิสูจน์ว่าไฟล์ไม่ถูกแก้
uploaded_by	VARCHAR(30)	ผู้อัปโหลด — id ผู้ใช้ของ ABS (`x-user-id`) · แถวที่ย้ายจาก K2 เก็บชื่อคน	ตอบว่าใครเอาไฟล์เข้ามา
uploaded_by_name	VARCHAR(200)	ชื่อผู้แนบ ณ ตอนแนบ (จาก `x-user-full-name` = ชื่อใน sps_auth.users)	หน้าจอแสดงชื่อจากคอลัมน์นี้ · เพิ่ม 2026-10-05 แทนการแปลงด้วย business_user ซึ่งเป็น id คนละชุด
uploaded_at	TIMESTAMP	เวลาที่อัปโหลด	เรียงลำดับไฟล์และใช้ตรวจสอบย้อนหลัง
deleted_flag	CHAR(1)	ธงลบแบบ soft delete — `CHAR(1)` ค่า `'Y'/'N'` (ไม่ใช่ boolean)	ลบแบบไม่ลบจริงเพื่อรักษาหลักฐาน · ทุก query ต้องกรอง `= 'N'` · ▲ ปัจจุบันยังไม่มี endpoint หรือปุ่มไหนตั้งค่าเป็น `'Y'` — อยู่ในข้อค้าง 2.5

sgi_compensation_histories

8 คอลัมน์

ประวัติการชดเชยรายร้าน (ข้ามเอกสาร)

มาจากไหน	K2 · migrate จาก ORA `FGI_IMPACT_STORE_COMPENSATE` + MSSQL `CompensateFlow` (แถวรอบก่อนหน้าของร้านเดียวกัน)
ใช้ทำอะไร	เก็บ ประวัติว่าร้านนี้เคยได้ชดเชยงวดไหน เท่าไร ส่งบัญชีเดือนไหน โดยไม่ผูกกับเอกสารฉบับใดฉบับหนึ่ง
ทำไมต้องมี	ผู้พิจารณาต้องเห็นประวัติย้อนหลังของร้านก่อนอนุมัติงวดใหม่ · และกฎ "ยอด 0 ติดกัน 3 เดือน → เดือนที่ 4 หยุดชดเชย" ต้องนับจากประวัตินี้
ใครเขียน	Job 6 / Job 8 หลังยอดถูกอนุมัติ · migration ตอน cutover
ใครอ่าน	ส่วน "ประวัติการชดเชย" ในหน้าเอกสาร · รายงาน

กติกาที่ต้องรู้ก่อนเขียนโค้ด

- `ref_doc_no` เป็น nullable เพราะประวัติที่ย้ายมาจากระบบเดิมบางแถวไม่มีเอกสารคู่กัน

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
id	BIGSERIAL	กุญแจแทน (surrogate key) ที่ฐานข้อมูลออกให้เอง เรียงเพิ่มทีละ 1	ใช้เป็น PK และเป็นเป้าของ FK จากตารางลูก · ห้ามเอาไปโชว์ผู้ใช้ (ผู้ใช้เห็น doc_no หรือรหัสร้านแทน) · เหตุผลที่ไม่ใช้ business key เป็น PK คือ business key ของระบบเดิมเปลี่ยนได้และบางตารางมีคีย์ผสม 3-4 คอลัมน์
store_code	VARCHAR(5)	รหัสร้านที่ประวัตินี้เป็นของ	แกนหลักในการดึงประวัติ — query ด้วยรหัสร้านไม่ใช่เลขเอกสาร
ref_doc_no	VARCHAR(10)	เลขเอกสารที่ทำให้เกิดยอดนี้ (ว่างได้)	ลิงก์กลับไปเอกสารต้นเรื่องเมื่อมี
submit_account_month	CHAR(7)	เดือนที่ส่งเข้าบัญชี (`YYYY-MM`)	ใช้กระทบยอดกับฝั่งบัญชีและเรียงประวัติ

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
compensate_amount	NUMERIC(14,2)	ยอดเงินที่ชดเชยงวดนั้น	ยอด 0 มีความหมาย — เป็นตัวนับว่าครบ 3 เดือนติดหรือยังตามกฎหยุดชดเชย
accounting_status	VARCHAR(30)	สถานะฝั่งบัญชีของยอดนี้	บอกว่ายอดนี้ไปถึงขั้นไหนของบัญชีแล้ว
external_ref	VARCHAR(100)	เลขอ้างอิงของระบบภายนอก (STA/Statement)	ใช้ตามยอดข้ามระบบเมื่อกระทบยอดไม่ตรง
created_at	TIMESTAMP	เวลาที่แถวนี้ถูกสร้าง (ตั้งอัตโนมัติด้วย DEFAULT CURRENT_TIMESTAMP)	ใช้เรียงลำดับเหตุการณ์ · เป็นฐานคำนวณอายุงานของ watchdog · ใช้สืบย้อนตอน incident ว่าแถวเกิดตอนไหน — ห้ามให้แอปเขียนทับ

sgi_document_cost_details

10 คอลัมน์

ยอดชดเชยแยกรายเดือน × รายร้านเปิดใหม่

มาจากไหน	K2 · migrate จาก MSSQL `ImpactCostDetail` (PK `ImpCostID`) · เพิ่มเข้าโครง 2026-08-06
ใช้ทำอะไร	เก็บ ยอดเป้าหมายและยอดจริงแยกตามเดือนและร้านเปิดใหม่ สำหรับกลุ่ม N และ NC
ทำไมต้องมี	ยอดรวมในหัวเอกสารตอบได้แค่ "เท่าไร" แต่ตอบไม่ได้ว่า "เดือนไหน ร้านไหน เท่าไร" · ฝั่งบัญชีและการตรวจสอบต้องการรายละเอียดระดับนี้
ใครเขียน	Job 8 ตอนสร้างเอกสาร · การคำนวณยอดในหน้าเอกสาร
ใครอ่าน	ตารางรายละเอียดค่าใช้จ่ายในหน้าเอกสาร · รายงานบัญชี

กติกาที่ต้องรู้ก่อนเขียนโค้ด

- 1 แถว = 1 (เอกสาร × ร้านเปิดใหม่ × ปี × เดือน)

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
id	BIGSERIAL	กุญแจแทน (surrogate key) ที่ฐานข้อมูลออกให้เอง เรียงเพิ่มทีละ 1	ใช้เป็น PK และเป็นเป้าของ FK จากตารางลูก · ห้ามเอาไปโชว์ผู้ใช้ (ผู้ใช้เห็น doc_no หรือรหัสร้านแทน) · เหตุผลที่ไม่ใช้ business key เป็น PK คือ business key ของระบบเดิมเปลี่ยนได้และบางตารางมีคีย์ผสม 3-4 คอลัมน์
doc_no	VARCHAR(10)	FK ไปหัวเอกสาร (NOT NULL · CASCADE)	มัดแถวเข้ากับเอกสาร
new_store_code	VARCHAR(5)	รหัสร้านเปิดใหม่ที่ยอดก้อนนี้เป็นของ	แยกยอดรายร้าน
cost_year	SMALLINT	ปี ค.ศ. ของยอดก้อนนี้	แกนเวลาแรก
cost_month	SMALLINT	เดือนของยอดก้อนนี้ (1–12)	แกนเวลาที่สอง — ทำให้ดูยอดรายเดือนได้
cost_target_n	NUMERIC(14,2)	ยอด เป้าหมาย กลุ่ม N	ฐานเปรียบเทียบของกลุ่ม N
cost_amount_n	NUMERIC(14,2)	ยอด จริง กลุ่ม N	ค่าที่เกิดขึ้นจริง เทียบกับเป้าหมาย
cost_target_nc	NUMERIC(14,2)	ยอด เป้าหมาย กลุ่ม NC	ฐานเปรียบเทียบของกลุ่ม NC

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
cost_amount_nc	NUMERIC(14,2)	ยอด จริง กลุ่ม NC	ค่าที่เกิดขึ้นจริงของกลุ่ม NC
created_at	TIMESTAMP	เวลาที่แถวนี้ถูกสร้าง (ตั้งอัตโนมัติด้วย DEFAULT CURRENT_TIMESTAMP)	ใช้เรียงลำดับเหตุการณ์ · เป็นฐานคำนวณอายุงานของ watchdog · ใช้สืบย้อนตอน incident ว่าแถวเกิดตอนไหน — ห้ามให้แอปเขียนทับ

sgi_document_running_numbers

4 คอลัมน์

ตัวนับเลขเอกสารต่อปี

มาจากไหน	K2 · migrate จาก MSSQL RunningNumber · เพิ่มเข้าโครง 2026-08-06
ใช้ทำอะไร	เก็บ เลขลำดับล่าสุดของเอกสารในแต่ละปี เพื่อออกเลข doc_no ถัดไป
ทำไมต้องมี	เลขเอกสารต้องเรียงต่อกันไม่ข้ามและไม่ซ้ำภายในปีเดียว · ถ้าใช้ MAX(running_no) + 1 จะเกิดเลขซ้ำทันทีเมื่อมีสองคนสร้างพร้อมกัน ตารางนี้ทำให้ล็อกแถวเดียว (SELECT ... FOR UPDATE) แล้วออกเลขได้อย่างปลอดภัย
ใครเขียน	Job 8 และ POST /sgi/document ตอนออกเลขเอกสาร
ใครอ่าน	เฉพาะตอนออกเลข — ไม่มีหน้าจอไหนอ่าน

กติกาที่ต้องรู้ก่อนเขียนโค้ด

- PK เป็น year — 1 แถวต่อ 1 ปี · ต้องล็อกแถวก่อนบวกเลขเสมอ
- ปีเป็น ค.ศ. ตามมติ 2026-08-06 (ทั้งระบบใช้ ค.ศ. ยกเว้นไฟล์ interface)

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
year	SMALLINT	ปี ค.ศ. ที่ตัวนับนี้เป็นของ	PK · ขึ้นปีใหม่ = แถวใหม่ เริ่มนับ 1 ใหม่
last_running_no	INTEGER	เลขลำดับล่าสุดที่ออกไปแล้วในปีนั้น	บวก 1 แล้วเอาไปประกอบเป็น doc_no = year/LPAD(no,5,'0')
updated_by	VARCHAR(30)	ผู้แก้ไขล่าสุด (รูปแบบเดียวกับ created_by)	ใช้ตอบคำถาม "ใครแก้ล่าสุด" ตอนข้อมูลเพี้ยน
updated_at	TIMESTAMP	เวลาที่แถวนี้ถูกแก้ล่าสุด	ใช้ตรวจว่าข้อมูล master ถูกแก้เมื่อไร และใช้เป็นตัวช่วยตอน sync/รีเฟรช cache ฝั่ง FE

โซน C Master ที่ SGI เป็นเจ้าของเอง — 4 ตาราง

sgi_impacted_stores

8 คอลัมน์

ทะเบียนร้านที่ถูกกระทบ (ร้าน SP ที่อยู่ในขอบเขตประกันรายได้)

มาจากไหน	K2 · migrate จาก ORA `FGI_IMPACT_STORE` (เลือกเฉพาะฝั่ง `_I` แบบ distinct) รวมกับ MSSQL `CompensateFlow.CompTransferSBPDate`
ใช้ทำอะไร	เก็บ รายชื่อร้านที่อยู่ในข่ายได้รับการชดเชย พร้อมพิกัดและผู้ดูแล (DV) เป็นสะพานเชื่อมสองระบบ: `store_code` ที่นี่คือค่าเดียวกับ `impacted_store_code` ทั้งโซน A และ B
ทำไมต้องมี	ตาราง `store/mas_store` ของระบบ SBP เดิมมีร้าน ทุกประเภท ไม่ได้บอกว่าร้านไหนเป็น SP ที่อยู่ในโครงการประกันรายได้ · ถ้าไม่มีตารางนี้ ทุก query ต้องไปไล่เงื่อนไข SP เองทุกครั้ง และไม่มีที่เก็บพิกัดที่ใช้วาดหมุด ALLMAP กับวันโอนเข้า SBP
ใครเขียน	Job 2 (ImportImpactStore) — upsert จาก ALLMAP · migration ตอน cutover
ใครอ่าน	Job 4/5/6/8 · `GET /sgi/document/{docNo}` (หมุดแผนที่) · หน้ารายการ/รายงาน

กติกาที่ต้องรู้ก่อนเขียนโค้ด

- PK เป็น `store_code` ตรง ๆ ไม่ใช่ surrogate — เพราะรหัสร้าน 5 หลักของ 7-Eleven เป็นค่าคงที่ตลอดอายุร้าน และทุกระบบอ้างด้วยรหัสนี้
- ไม่เก็บชื่อร้าน/ที่อยู่ซ้ำ — อ่านจาก `store/mas_store` ของระบบเดิมเสมอ เก็บซ้ำเมื่อไรข้อมูลจะเพี้ยนกันทันที

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
store_code	VARCHAR(5)	รหัสร้าน 5 หลักของร้านที่ถูกกระทบ (ร้าน SP)	PK · ใช้ join ไป `store/mas_store` ของระบบเดิมเพื่อเอาชื่อ/ที่อยู่ · เป็นค่าเดียวกับ `impacted_store_code` ในโซน A/B
dv_code	VARCHAR(20)	รหัสเขต (District Vendor) ที่ร้านนี้สังกัด	ใช้จัดกลุ่มรายงานรายเขต และเป็นตัวกรองของอีเมลสรุปราย DV (EM-06)
opt_dv_user_id	VARCHAR(30)	รหัสผู้ใช้ของ DV ที่ดูแลร้านนี้	ใช้เป็น เงื่อนไข gate ของ Job 8b: ถ้าว่าง = เปิด workflow ไม่ได้ เพราะไม่รู้จะมอบงานให้ใคร (ตั้ง `workflow_generation_status = 'N'`)
latitude	NUMERIC(10,7)	ละติจูดของร้าน (ทศนิยม 7 ตำแหน่ง)	ใช้วาดหมุดบนแผนที่ ALLMAP ในหน้าเอกสาร · ความละเอียด 7 ตำแหน่ง ≈ 1 ซม. เพียงพอสำหรับรัศมี 1–2 กม.
longitude	NUMERIC(10,7)	ลองจิจูดของร้าน	คู่กับ `latitude` — ใช้คำนวณ/ตรวจระยะรัศมีกระทบ 1 กม. (กทม.) / 2 กม. (ต่างจังหวัด)
transfer_sbp_date	DATE	วันที่ร้านโอนเข้าเป็นร้าน SBP	ใช้ตัดสินว่าร้าน อายุน้อยกว่า 12 เดือน 15 วันหรือไม่ ซึ่งเป็นเงื่อนไข pre-accept ของ Job 5 · nullable เพราะร้านเก่าบางร้านไม่มีวันที่นี้ในระบบเดิม
is_active	BOOLEAN	ธงเปิด/ปิดการใช้งานของแถว master	ปิดแทนการลบ เพราะเอกสารเก่าที่อ้างรหัสนี้ต้องยังแสดงชื่อได้ · dropdown ฝั่ง FE ต้องกรอง `is_active = TRUE` แต่หน้าอ่านเอกสารเก่าห้ามกรอง
updated_at	TIMESTAMP	เวลาที่แถวนี้ถูกแก้ล่าสุด	ใช้ตรวจว่าข้อมูล master ถูกแก้เมื่อไร และใช้เป็นตัวช่วยตอน sync/รีเฟรช cache ฝั่ง FE

sgi_external_factors

5 คอลัมน์

master ปัจจัยภายนอก

มาจากไหน	K2 · migrate จาก MSSQL `FactorProfile` — หน้าจอ `k2-factors.html` (SRS 3.1.9)
ใช้ทำอะไร	เก็บ รายการปัจจัยภายนอกที่ใช้อธิบายว่าทำไมยอดขายตก เช่น ก่อสร้าง/ปิดถนน/น้ำท่วม ให้ผู้ใช้เลือกจาก dropdown แทนการพิมพ์เอง
ทำไมต้องมี	ถ้าให้พิมพ์เป็นข้อความอิสระ จะสรุปรายงานว่า "ปัจจัยไหนทำให้ชดเชยบ่อยที่สุด" ไม่ได้เลย เพราะคนละคนพิมพ์คนละแบบ
ใครเขียน	หน้า `k2-factors.html` ผ่าน `POST/PUT/DELETE /sgi/master/factors`
ใครอ่าน	dropdown ในหน้าเอกสาร · `sgi_document_external_factors` อ้างด้วย FK

กติกาที่ต้องรู้ก่อนเขียนโค้ด

- ลบจริงไม่ได้ถ้ามีเอกสารอ้างอยู่ — ใช้ `is_active = FALSE` แทน

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
factor_code	VARCHAR(30)	รหัสปัจจัย เช่น `F003`	PK · เป็นค่าที่ `sgi_document_external_factors.factor_code` อ้างถึง
factor_name	VARCHAR(200)	ชื่อปัจจัยภาษาไทยที่ผู้ใช้เห็น เช่น "การก่อสร้าง / ปิดถนน"	แสดงใน dropdown และในรายงาน
factor_remark	VARCHAR(500)	คำอธิบายเพิ่มเติมของปัจจัย	ช่วยให้ผู้ใช้เลือกถูกตัว — ไม่ได้ใช้ในการคำนวณใด ๆ
is_active	BOOLEAN	ธงเปิด/ปิดการใช้งานของแถว master	ปิดแทนการลบ เพราะเอกสารเก่าที่อ้างรหัสนี้ต้องยังแสดงชื่อได้ · dropdown ฝั่ง FE ต้องกรอง `is_active = TRUE` แต่หน้าอ่านเอกสารเก่าห้ามกรอง
updated_at	TIMESTAMP	เวลาที่แถวนี้ถูกแก้ล่าสุด	ใช้ตรวจว่าข้อมูล master ถูกแก้เมื่อไร และใช้เป็นตัวช่วยตอน sync/รีเฟรช cache ฝั่ง FE

sgi_section_owners

10 คอลัมน์

ทะเบียนผู้รับผิดชอบต่อขั้น (× ภาค)

มาจากไหน	K2 · `CompenOrganizeProfile` (Section × Zone · 22 แถวจริง) + วิว `v_fgi_sbp_approver` ที่ batch job เดิมใช้เติมผู้อนุมัติ
ใช้ทำอะไร	บอกว่า ขั้นไหน (× ภาคไหน) มีใครเป็นผู้รับผิดชอบ — ใช้ตอนมอบหมายงานขั้นถัดไป
ทำไมต้องมี	ระบบใหม่ส่งงานต่อด้วยกติกา *เจ้าของงานคนเดิมของร้านนั้น* (SDD GI · เคสต่อเนื่อง) ซึ่งตอบไม่ได้เมื่อเป็น เอกสารใบแรกของร้าน เพราะยังไม่มีประวัติ ระบบเดิมแก้ด้วยทะเบียนนี้ และเติมให้เฉพาะตอนที่ผู้อนุมัติยังว่าง (WHERE sbp_dv_emp_id1 IS NULL OR …) ตารางนี้จึงเป็น fallback ไม่ใช่ตัวตัดสินหลัก
ใครเขียน	ตั้งค่าโดยผู้ดูแลระบบ (ยังไม่มีหน้าจอ — seed/แก้ผ่าน SQL)
ใครอ่าน	`resolveNextApprover` ของ `POST /sgi/document/{docNo}/actions` เมื่อหาเจ้าของงานคนเดิมไม่เจอ

กติกาที่ต้องรู้ก่อนเขียนโค้ด

- เลือกแถวจากเจาะจงมากไปน้อย: (ภาค+ประเภท) → (ภาค) → (ประเภท) → (ทุกภาคทุกประเภท) แล้วจึงเรียงด้วย `priority` — ล้อการไล่ระดับของวิวเดิม (`NVL(NVL(NVL(dv, acting_dv), gm), vp)`)
- ลำดับที่ BE ใช้: เจ้าของงานคนเดิมของร้านนั้น → ทะเบียนนี้ → 400 (ของเดิม `catch { log }` แล้วเดินต่อเงียบ ทำให้เอกสารไม่ถูกสร้างโดยไม่มีใครรู้สาเหตุ)
- ▲ `section_code` มีแค่ 5 ขั้นของ flow ใหม่ — ไม่มี `07` และไม่มี `04/05` ที่ SDD v7.5 ตัดไปแล้ว (ทะเบียนเดิม 22 แถวมี `04` · `07` อยู่ จึงยกมาใช้ตรง ๆ ไม่ได้)
- ▲ `user_id` คือ id ของ ABS (`sps_auth.users.id`) ไม่ใช่ `business_user.user_id` — สองตารางให้ id ไม่ตรงกัน และ `x-user-id` ที่ระบบใช้มาจาก ABS

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
id	BIGSERIAL	กุญแจแทน (surrogate key) ที่ฐานข้อมูลออกให้เอง เรียงเพิ่มทีละ 1	ใช้เป็น PK และเป็นเป้าของ FK จากตารางลูก · ห้ามเอาไปโชว์ผู้ใช้ (ผู้ใช้เห็น doc_no หรือรหัสร้านแทน) · เหตุผลที่ไม่ใช้ business key เป็น PK คือ business key ของระบบเดิมเปลี่ยนได้และบางตารางมีคีย์ผสม 3-4 คอลัมน์
section_code	VARCHAR(2)	ขั้นของ workflow	`06` `08` `01` `02` `03` เท่านั้น (CHECK)
zone_code	VARCHAR(10)	ภาคของร้าน	NULL = ทุกภาค — แบบเดียวกับ section 04/06/08 ของทะเบียนเดิม ส่วน 07 เดิมแยกรายภาค 14 แถว
store_type	VARCHAR(10)	ประเภทร้าน	NULL = ทุกประเภท · ของเดิมจับคู่ด้วย ภาค + ประเภทร้าน (`zone_i` + `branchtype_i`) ไม่ใช่ภาคอย่างเดียว · วัดจริง 429 แถว: ในโซนเดียวกันทุกประเภทได้คนเดียวกัน ยกเว้น B (ร้านบริษัท) ที่ไม่มีผู้ดูแลฝั่ง SBP ทั้ง 13 โซน
user_id	BIGINT	ผู้รับผิดชอบ	`sps_auth.users.id` (ABS)
priority	INT	ลำดับการเลือก	เลขน้อยถูกเลือกก่อน — ขั้นเดียวมีหลายคนได้ (section 08 เดิมมี 5 คน)
is_active	BOOLEAN	เปิด/ปิดใช้งาน	ปิดแทนการลบ เพื่อให้ตามรอยย้อนหลังได้
remark	VARCHAR(500)	หมายเหตุ	ไม่มีผลต่อ logic
updated_by	VARCHAR(100)	ผู้แก้ไขล่าสุด (รูปแบบเดียวกับ `created_by`)	ใช้ตอบคำถาม "ใครแก้ล่าสุด" ตอนข้อมูลเพี้ยน

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
updated_at	TIMESTAMP	เวลาที่แถวนี้ถูกแก้ล่าสุด	ใช้ตรวจว่าข้อมูล master ถูกแก้เมื่อไร และใช้เป็นตัวช่วยตอน sync/รีเฟรช cache ฝั่ง FE

sgi_competitors

6 คอลัมน์

master แบรนด์คู่แข่ง

มาจากไหน	K2 · หน้าจอ `k2-competitors.html` (คัดลอกจากหน้าจอ K2 เดิมแบบตรงตัว 12 แบรนด์ รหัส 01–12)
ใช้ทำอะไร	เก็บ รายชื่อแบรนด์ร้านคู่แข่ง (Lotus's Express · CJ Express · ฯลฯ) ทั้งภาษาไทยและอังกฤษ
ทำไมต้องมี	ข้อมูลคู่แข่งที่ดึงจาก ALLMAP มาเป็น รายสาขา (มีชื่อสาขา พิกัด วันเปิด) แต่ต้องรู้ว่าสาขานั้นเป็นแบรนด์อะไรเพื่อสรุปรายงาน ตารางนี้คือชั้นแบรนด์
ใครเขียน	หน้า `k2-competitors.html` ผ่าน `POST/PUT/DELETE /sgi/master/competitors`
ใครอ่าน	dropdown "ร้านคู่แข่ง (Master)" ในหน้าเอกสาร · `sgi_document_competitors` · `sgi_fgi_impact_competitors`

กติกาที่ต้องรู้ก่อนเขียนโค้ด

- ▲ อย่าสับสนกับ `sgi_document_competitors` — ตารางนี้คือ *แบรนด์* (11 แถว) ส่วนอีกตารางคือ *สาขาของคู่แข่งที่กระทบเอกสารฉบับหนึ่ง* (หลายแถวต่อเอกสาร)

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
competitor_code	VARCHAR(30)	รหัสแบรนด์ `01–11`	PK · ถูกอ้างเป็น `brand_code` ในตารางคู่แข่งระดับสาขาทั้งสองตัว — คนละรหัสกับ `competitor_store_code` ที่ ALLMAP ส่งมา
name_th	VARCHAR(200)	ชื่อแบรนด์ภาษาไทย	แสดงในหน้าจอและรายงานทั้งหมด
name_en	VARCHAR(200)	ชื่อแบรนด์ภาษาอังกฤษ	ใช้ตอน export และตอน match ชื่อที่ได้จาก ALLMAP ซึ่งมาเป็นอังกฤษ
remark	VARCHAR(500)	หมายเหตุของแบรนด์	บันทึกช่วยจำ ไม่มีผลต่อ logic
is_active	BOOLEAN	ธงเปิด/ปิดการใช้งานของแถว master	ปิดแทนการลบ เพราะเอกสารเก่าที่อ้างรหัสนี้ต้องยังแสดงชื่อได้ · dropdown ฝั่ง FE ต้องกรอง `is_active = TRUE` แต่หน้าอ่านเอกสารเก่าห้ามกรอง
updated_at	TIMESTAMP	เวลาที่แถวนี้ถูกแก้ล่าสุด	ใช้ตรวจว่าข้อมูล master ถูกแก้เมื่อไร และใช้เป็นตัวช่วยตอน sync/รีเฟรช cache ฝั่ง FE

สรุปความครบถ้วน

ตารางที่อธิบายไว้	21 ตาราง — ครบทุกตารางที่ SGI ต้องสร้างใหม่
คอลัมน์ที่อธิบายไว้	270 คอลัมน์ — ครบทุกคอลัมน์ใน DDL
คอลัมน์ที่ยังไม่มีคำอธิบาย	0 — กฎ #88 ใน tools/check_docs.py บังคับให้เป็น 0
เอกสารนี้ตกยุคได้ไหม	ไม่ได้ — รายชื่อคอลัมน์อ่านสดจาก DDL ถ้า schema เพิ่มคอลัมน์แล้วยังไม่มีคำอธิบาย ชุดตรวจจะไม่ผ่านทันที